From Episodes to Revisable Concepts

The Experience Learning Layer for Evidence-Grounded Learning in Language Agents

Danny Ruchtie

2026-08-09

Table of contents

Abstract

Living working draft v0.6 - 9 August 2026

Version 0.6 sharpens the contribution boundary after a review of current agent-memory, experience-learning, provenance, belief-revision, self-scaffolding, and evaluation research. It defines one confirmatory outcome, explicit success and failure gates, a benchmark-first implementation plan, a narrower ELL-Core, and a separately gated programme for learning how ELL itself learns. Earlier drafts remain available through Git history.

Research status. This document is a working architecture paper and preregistered evaluation plan. It defines the proposed system, hypotheses, data model, implementation contract, and experiments. It does not report empirical results. Results will be added only after the open implementation and evaluation harness can reproduce them.

Large language model agents can store and retrieve previous interactions, yet retrieval alone does not establish that an agent has learned from experience. Learning requires a system to identify patterns across episodes, distinguish observations from hypotheses, form reusable concepts, apply those concepts in new situations, and revise them when later evidence disagrees. Recent agent-memory research has introduced reflection, experience-derived insights, workflow induction, dynamic memory graphs, semantic and procedural memory, schema induction, and temporally grounded updates. The remaining challenge is therefore not simply to add another memory store, but to make the transition from experience to general knowledge explicit, inspectable, revisable, and directly measurable.

This paper proposes the Experience Learning Layer (ELL), an open, model-independent architecture positioned between an agent's experience history and its decision process. ELL preserves raw episodes in an append-only store, creates typed associations between them, represents reflections as provisional interpretations, and promotes compatible reflections into versioned concepts. Each concept records its scope, supporting evidence, counterevidence, confidence, temporal validity, lineage, and observed utility. Concepts can be corroborated, contested, revised, superseded, or retired without erasing their evidential history.

The system north star is to give any AI a natural, evolving intuition about the user or organisation: grounded in experience, economical to use, sensitive to change, explainable when inspected, and always under user control.

Here intuition is neither anthropomorphism nor a new memory category. It names an emergent system capability produced by evidence-grounded compression, prediction, association, relevance selection, temporal adaptation, and outcome feedback. At use time, ELL should supply a compact intuition packet rather than dump raw history into a model, while retaining links that permit inspection, correction, deletion, and uncertainty-aware expansion.

The paper contributes a minimal typed architecture, a concept-lifecycle protocol, an evidence ledger, a controlled benchmark, and an evaluation-first implementation plan. It also specifies a follow-on governed self-scaffolding programme in which versioned learning strategies may improve reflection, consolidation, retrieval, and error recovery while an immutable outer constitution retains authority over evidence, consent, permissions, deletion, canonical commits, and evaluation. It does not claim that reflection, abstraction, provenance, temporal memory, external memory,

or self-scaffolding is individually novel. Its primary confirmatory claim remains narrower: under matched model and total-compute conditions, an evidence-governed concept layer should improve transfer to structurally related but lexically different tasks over the strongest eligible non-parametric baseline, while satisfying preregistered guardrails for unsupported generalisation, evidence quality, adaptation, governance, and cost. If it does not, the concept layer is not justified and the self-scaffolding extension does not proceed.

Keywords: agent memory; experiential learning; reflection; concept formation; semantic memory; continual learning; self-scaffolding; meta-learning; provenance; language agents; open source

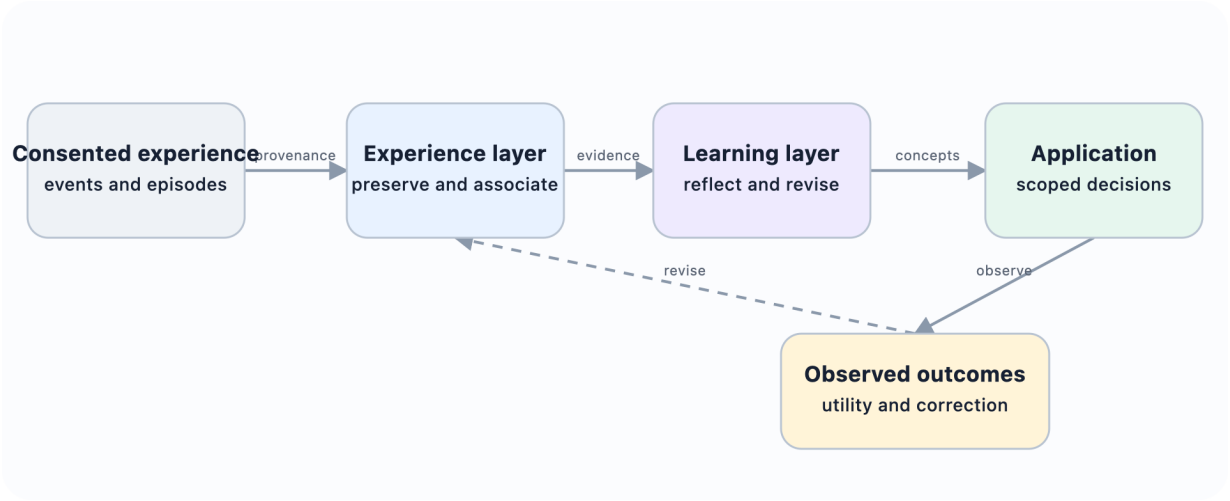

Figure 1: Where the Experience Learning Layer sits

1 Introduction

A language agent can remember an earlier event without learning anything general from it. A retrieval system may surface five examples of the same failure, yet the agent may still repeat that failure because the episodes have never been converted into a reusable and appropriately scoped principle. This distinction matters as agents move from single-turn assistance toward persistent collaboration, long-running tasks, and multi-session interaction.

The intended user experience is stronger than factual continuity. An AI should understand conversational shorthand, surface relevant context just in time, generalise usefully across related situations, notice when a previous pattern has changed, correct itself rapidly, and express calibrated uncertainty. Explanations should be available when requested without forcing provenance detail into every interaction. These behaviours constitute the proposed operational meaning of an evolving intuition; they do not imply consciousness, personhood, or a simulation of human cognition.

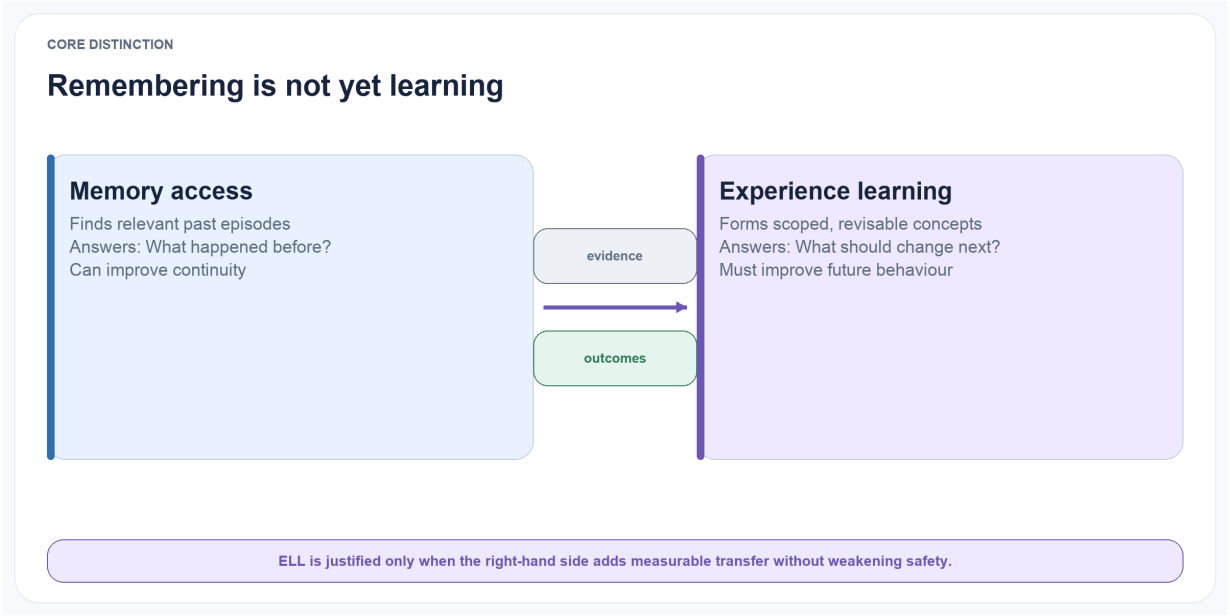

Figure 1.1: **Remembering is not yet learning.** Memory access can improve continuity, but ELL is justified only if evidence and outcomes produce scoped concepts that measurably improve future behaviour.

The first generation of agent-memory systems established that external memory could improve continuity beyond a model's context window. Generative Agents stored observations, ranked them for retrieval, and periodically synthesised higher-level reflections that influenced later plans (Park et al. 2023). Reflexion used textual feedback about prior attempts as a non-parametric learning signal (Shinn et al. 2023). ExpeL extracted natural-language insights from collections of task experiences and reused both insights and successful trajectories at inference time (Zhao et al. 2024). MemGPT approached the problem as virtual context management, allowing an agent to move information between limited working context and external storage (Packer et al. 2023).

CoALA organised these developments within a broader cognitive architecture containing working, episodic, semantic, and procedural memory (Sumers et al. 2024).

Subsequent systems have increasingly abstracted experience rather than merely retrieving it. Voyager accumulated reusable executable skills (G. Wang et al. 2023). Agent Workflow Memory induced recurring action routines from prior trajectories (Z. Z. Wang et al. 2025). A-MEM dynamically linked and revised memory notes as new information arrived (Xu et al. 2025). Reflective Memory Management used prospective and retrospective reflection to improve memory granularity and retrieval (Z. Tan et al. 2025). Hindsight separates world, experience, observation, and opinion networks and allows opinions to evolve with confidence (Latimer et al. 2026). More recent architectures explicitly construct schemas, concepts, semantic knowledge, or procedural knowledge from episodes, including DCPM, GAAMA, and PlugMem (Fei et al. 2026; Paul, Sharma, and Sareen 2026; Yang et al. 2026). A recent survey describes this field-wide progression as storage, reflection, and experience abstraction (Luo et al. 2026).

This rapid progress changes the research question. It is no longer defensible to claim that transforming episodes into abstractions is itself novel. Nor are provenance, non-monotonic belief revision, temporal validity, or dependency-based retraction new in isolation. The more precise question is: Does combining these ideas in an external language-agent learning layer produce a measurable advantage over simpler memory methods, and can that advantage survive explicit safety, adaptation, and cost constraints?

ELL addresses this question by separating four objects that are often collapsed into one textual memory:

1. An episode records what occurred in a specific context.
2. A reflection is a provisional interpretation of one or more episodes.
3. A concept is a reusable proposition or strategy supported by multiple reflections and episodes.
4. An application outcome records whether using a concept helped in a later situation.

The separation is important because a plausible explanation is not automatically reliable knowledge. Suppose a project proposal is rejected after stakeholders were consulted late. A reflection may state that late stakeholder involvement caused the rejection. That is a useful hypothesis, but a single episode does not justify a universal rule. A concept should only be promoted after corroborating cases, explicit checks for counterexamples, and a scope statement such as "cross-functional initiatives with external implementation dependencies." New evidence may later narrow, challenge, or replace it.

ELL treats every concept as a versioned claim rather than a timeless fact. The system retains the episodes that support it, episodes that contradict it, the contexts in which it has been applied, and the outcomes of those applications. This makes concept formation inspectable and enables evaluation at the level where learning is claimed to occur.

The architecture is inspired by, but does not attempt to reproduce, human memory. Tulving's distinction between episodic and semantic memory motivates the separation between event-specific records and general knowledge (Tulving 1972). Complementary Learning Systems theory motivates a fast episodic path and a slower consolidation path that integrates common structure across experiences (McClelland, McNaughton, and O'Reilly 1995; Kumaran, Hassabis, and McClelland 2016). Bartlett's account of reconstructive remembering provides a warning as much as an inspiration: abstraction can impose existing schemas on ambiguous experience and thereby distort it (Bartlett 1932). For this reason, ELL preserves provenance, counterevidence, and reversible revisions.

1.1 Scope

ELL is an external learning layer. It does not update the parameters of the underlying language model. It does not prescribe how raw activity is captured from calendars, files, screens, sensors, or chat applications. It starts after an application has produced a canonical stream of consented experience records. It is designed to support conversation histories, task trajectories, product-work records, and other event streams through a shared schema.

The confirmatory research scope is deliberately narrower than a complete personal-intelligence system. ELL-Core contains only source evidence, episodes, provisional reflections, versioned concepts, evidence links, application receipts, and outcomes. It focuses on:

- association between episodes;
- reflection over individual and grouped episodes;
- concept formation and consolidation;
- application of concepts to later decisions;
- revision after new evidence and outcomes;
- direct evaluation of concept quality and transfer.

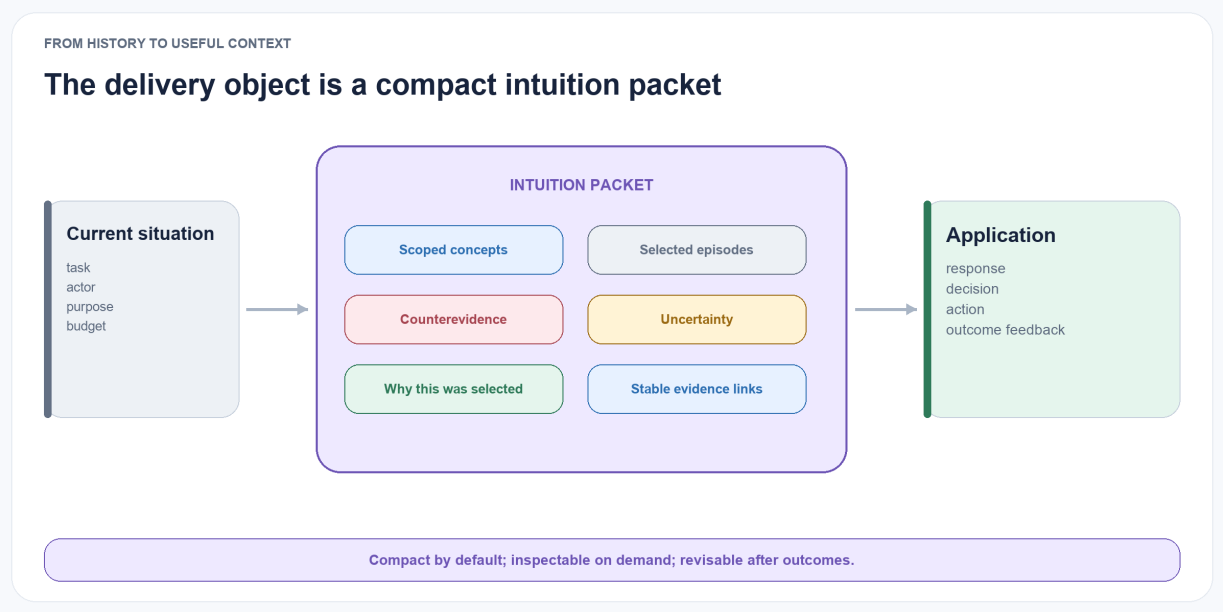

Figure 1.2: **The intended delivery object.** ELL supplies a compact intuition packet containing scoped concepts, selected episodes, counterevidence, uncertainty, selection reasons, and stable links for deeper inspection.

Perception, multimodal capture, large-scale identity resolution, autonomous skill evolution, learned memory-operation policies, neural memory, production provider integrations, and parametric fine-tuning are outside the first confirmatory study. Governed self-scaffolding is nevertheless specified as a first-class follow-on programme: ELL may eventually learn versioned strategies for reflection, consolidation, retrieval, and error recovery, but only after ELL-Core clears its success gates and without granting a learned strategy authority over evidence, consent, permissions, deletion, or canonical commits.

1.2 Contributions

This working paper specifies six intended contributions.

A minimal experience-to-concept contract. It defines distinct schemas and operations for evidence, episodes, reflections, concepts, applications, and outcomes without requiring a graph, vector database, or hosted memory provider.

An evidence-grounded concept lifecycle. Concepts move through explicit states—proposed, corroborated, contested, revised, superseded, and retired—while retaining lineage and provenance.

A separation between confidence and eloquence. Concept confidence is computed from observable evidence, contradiction, diversity, and outcome history rather than accepted directly from an LLM's self-reported certainty.

A confirmatory evaluation protocol. One primary transfer outcome is paired with preregistered epistemic, adaptation, governance, and cost guardrails. Component metrics diagnose why the result occurred; they do not substitute for behavioural improvement.

An evaluation-first open implementation plan. The benchmark and simple baselines are built before the concept engine. Versioned releases will include schemas, prompts, configurations, generators, evaluation scripts, raw receipts, and paper sources under permissive licences, with reproducibility runs using open-weight models.

A governed self-scaffolding research programme. A slower meta-learning loop treats the strategy used to learn and apply concepts as a typed, versioned, replayable object. Candidate strategies must beat a frozen policy under matched conditions, pass immutable governance gates, survive shadow and canary evaluation, and retain a complete rollback path before promotion.

2 Problem Definition

2.1 Experience learning

Let an agent encounter an ordered experience stream E={e1 ,e2 ,…,en}.

Each episode ei contains a time-bounded record of context, observation, action, outcome, source, and metadata. Given this stream, an experience-learning system should produce a set of concepts C that improve decisions on future situations while remaining grounded in the episodes from which they were induced.

The goal is not to compress the entire history into a shorter text. Compression can preserve frequent information while losing exceptions, causal uncertainty, temporal change, and source identity. The desired output is a set of claims or strategies with explicit conditions and evidence.

A concept is useful only if it satisfies several properties:

- Grounded: its support can be traced to specific episodes and reflections.
- General: it applies beyond a single remembered case.
- Scoped: it states where it is expected to apply and where it may not.
- Revisable: contradictory evidence can change its status or content.
- Actionable: it can affect a later response, plan, or decision.
- Measurable: its correctness and utility can be evaluated independently.

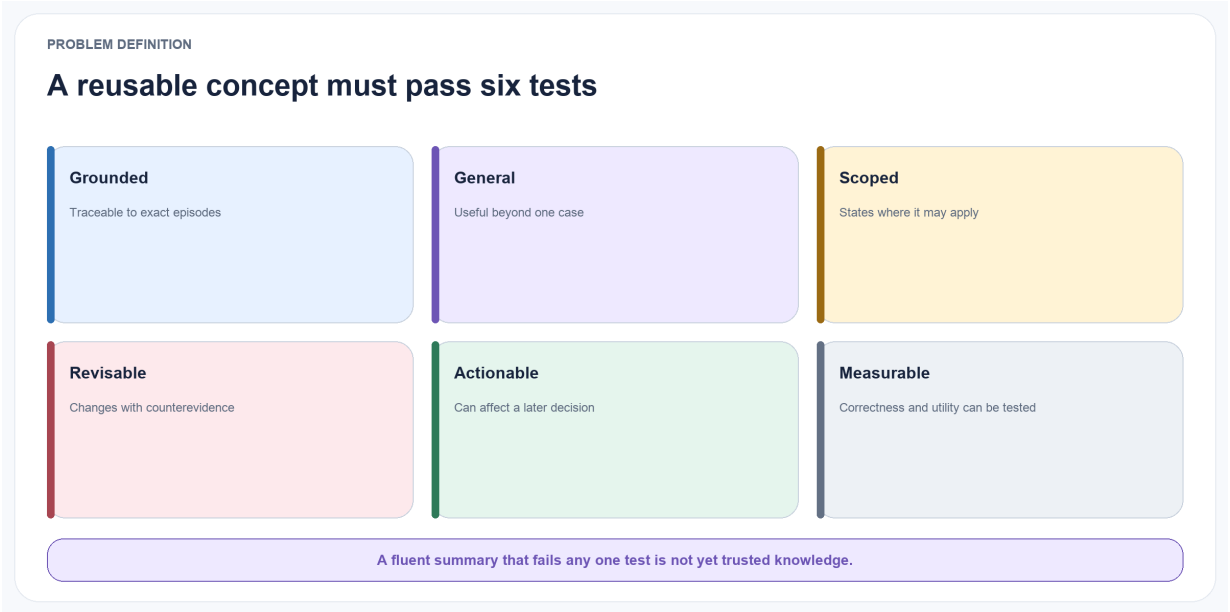

Figure 2.1: **Six tests for a useful concept.** Grounding and generality are necessary but insufficient: a durable concept must also declare scope, remain revisable, affect action, and support independent measurement.

2.2 Failure modes addressed

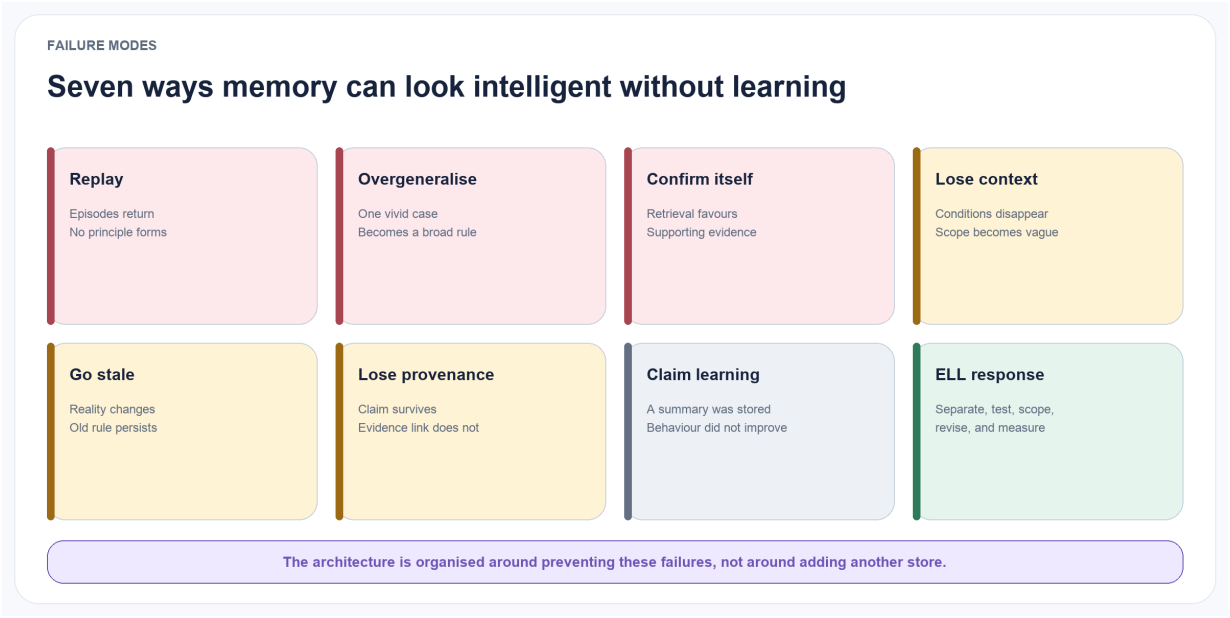

Figure 2.2: **Memory can appear capable without learning.** These failure modes motivate the separation of episodes, reflections, concepts, evidence, and outcomes.

ELL is designed around seven common failure modes.

Episode replay without abstraction. Relevant cases are retrieved, but no reusable principle is formed.

Premature generalisation. A single salient episode becomes a broad rule.

Self-confirming reflection. Once a reflection is stored, later retrieval favours evidence that agrees with it.

Context collapse. A concept loses the conditions under which it was valid.

Temporal staleness. New evidence changes what is true, but the old concept continues to guide behaviour.

Provenance loss. A generated statement can no longer be connected to the experience that justified it.

Behavioural ambiguity. A system claims to have learned because it stored a summary, without showing improved transfer or decision quality.

2.3 Research questions and claim hierarchy

The confirmatory ELL-Core study will answer five questions, but they do not have equal confirmatory status. A sixth question defines a separate follow-on study rather than enlarging the first comparison.

RQ1 — Separation. Does representing reflections as provisional objects before concept consolidation reduce unsupported generalisation compared with direct episode-to-rule summarisation?

RQ2 — Revision. Do explicit counterevidence, temporal validity, and versioned lifecycle states improve adaptation when the underlying pattern changes?

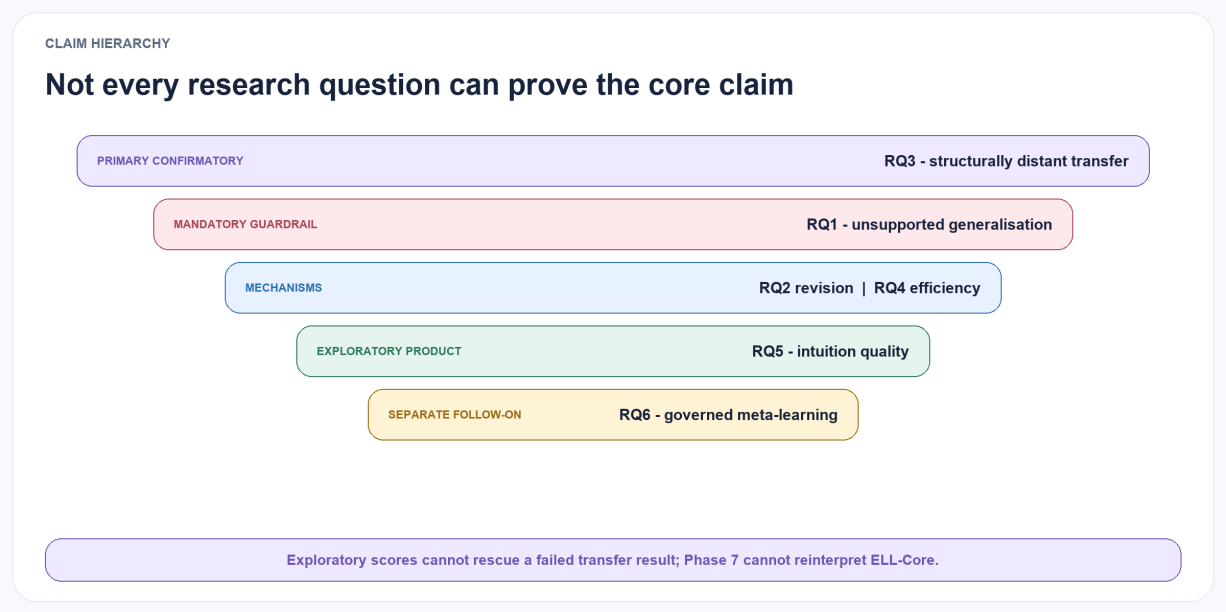

Figure 2.3: **The claim hierarchy prevents scope drift.** Transfer is primary, unsupported generalisation is a mandatory guardrail, revision and efficiency are mechanisms, intuition quality is exploratory, and self-scaffolding is a separate follow-on claim.

RQ3 — Transfer. Do consolidated concepts improve performance on new situations that share latent structure but differ in wording and surface details?

RQ4 — Efficiency. Can compact concepts plus selected source episodes match or improve downstream performance while using less retrieval context than episode-only methods?

RQ5 — Intuition quality. Can a compact, uncertainty-aware intuition packet improve shorthand understanding, just-in-time relevance, change detection, and rapid correction compared with no-memory, raw-history, and ordinary retrieval-augmented generation baselines?

RQ6 — Governed meta-learning. After ELL-Core has been validated, can a versioned self-scaffolding controller improve time-forward task utility, adaptation, or efficiency over the strongest frozen learning scaffold without increasing unsupported generalisation, negative transfer, privacy failure, or governance violations?

RQ3 is the primary confirmatory question. RQ1 is its epistemic safety guardrail: a transfer gain does not count if it is purchased through materially greater unsupported generalisation. RQ2 and RQ4 are secondary mechanism and efficiency questions. RQ5 is an exploratory product-facing question and cannot establish the core claim by itself. RQ6 is activated only in the separately preregistered self-scaffolding phase; it cannot improve, reinterpret, or rescue the ELL-Core verdict.

2.4 Hypotheses

H1. ELL will produce higher concept-scope accuracy and lower overgeneralisation than a direct insight-extraction baseline.

H2. ELL will revise or contest invalid concepts more quickly after contradictory evidence than append-only reflection or rolling-summary baselines.

H3. ELL will yield a positive transfer gain on held-out tasks whose latent rule is represented in prior experiences, even when lexical similarity to those experiences is low.

H4. Retrieving concepts together with a small number of supporting episodes will reduce median input tokens per decision without reducing task success.

H5. Compact intuition packets will improve shorthand resolution, just-in-time context precision, change detection, and calibrated correction over no-memory, raw-history, and ordinary RAG baselines at an equal retrieval budget.

H6. ELL will reduce end-to-end decision tokens, latency, and measured hardware-time or energy proxies while preserving or improving task utility; this hypothesis includes the cost of background association, reflection, and consolidation rather than treating those operations as free.

H7. Among scaffold candidates that satisfy every immutable governance gate, a governed self-scaffolding controller will improve time-forward utility per total cost over the strongest frozen scaffold while preserving calibration, counterevidence recall, correction performance, and rollback reliability. This is a follow-on hypothesis and is not tested in the first confirmatory study.

2.5 Primary estimand

The primary estimand is the paired absolute difference in task success between full ELL-Core and the strongest eligible baseline on sealed transfer cases whose latent rule was present in the experience stream but whose wording and surface structure differ from the supporting episodes. Eligibility requires the same source stream, base model, response budget, retrieval opportunity, and outcome information. Total cost includes ingestion, embedding, reflection, validation, consolidation, retrieval, and answer generation.

Concept accuracy, citation quality, and calibration remain necessary diagnostics, but they cannot rescue a failed behavioural result. Conversely, a transfer improvement does not count as success unless the overgeneralisation, evidence, adaptation, governance, replication, and cost gates in Section 11 also pass.

These hypotheses are weakened or rejected if the primary confidence interval includes no improvement, if the practically meaningful effect threshold is missed, if concept-level gains fail to translate into behaviour, or if the added architecture violates a guardrail. This hierarchy prevents exploratory intuition scores or a favourable component metric from being reported as proof of learning.

3 Related Work and Positioning

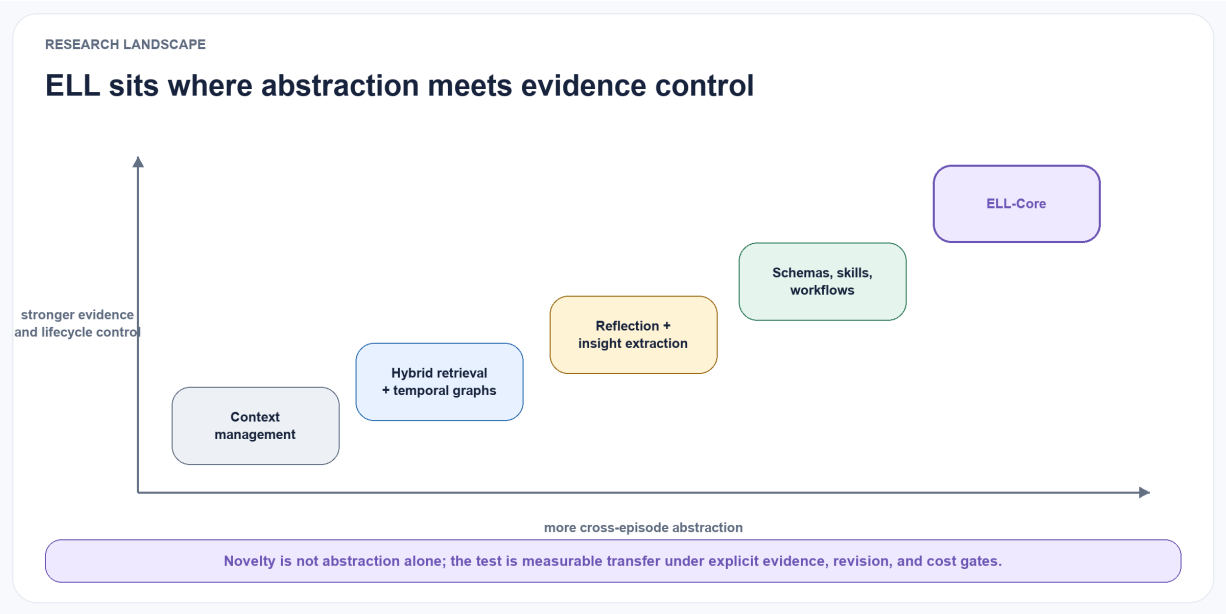

Figure 3.1: **ELL's position in the research landscape.** Existing work spans context management, hybrid retrieval, reflection, temporal graphs, schemas, skills, and workflows. ELL's narrower claim sits where cross-episode abstraction meets strong evidence and lifecycle control.

3.1 Production memory and context infrastructure

Production-oriented systems establish useful implementation patterns without settling ELL's epistemic model.

MemGPT, now developed as Letta, treats a finite context window as a managed hierarchy and gives the agent explicit operations for moving information between tiers (Packer et al. 2023; Letta 2026). Mem0 provides a developer-facing memory layer that extracts, consolidates, and retrieves salient conversational information, with an optional graph representation (Chhikara et al. 2025). Zep and its open-source Graphiti engine maintain temporally valid entities and relations, preserve episode provenance, and combine semantic, lexical, and graph retrieval (Rasmussen et al. 2025; Zep 2026). Hindsight separates world, experience, observation, and opinion networks, combining vector, keyword, graph, and temporal retrieval with evolving confidence-bearing opinions (Latimer et al. 2026).

TencentDB Agent Memory has expanded from layered conversational memory into a team-oriented memory hub. Its public design includes L0 conversations, L1 atoms, L2 scenarios, and L3 core or persona representations, alongside versioned skills, Wiki, CodeGraph, ownership, status, visibility, agent bindings, access-control lists, and budgeted BM25, vector, and reciprocal-rank-fusion retrieval (TencentDB Agent Memory Team 2026). These features overlap with several

earlier ELL design goals. TencentDB must therefore be treated as a substantive comparison system, not described merely as a raw-memory store.

TurboVec belongs at a different layer. It is a local compressed vector index built on TurboQuant, with online ingestion and query-time allow-list filtering; it does not define memory semantics, concept lifecycle, or learning policy (Shukla, Pandey, and Tiwari 2026). ELL may evaluate it as a `RetrievalCandidateProvider` against exact search, HNSW, and other approximate indexes, but it is not an end-to-end memory-system baseline.

ELL adopts four principles from this family: explicit context-budget management; incremental temporal updates; hybrid candidate retrieval; and a drill-down path from compact context to source episodes. The corresponding integration points are replaceable ContextManager, MemoryProjection, TemporalRelationIndex, and RetrievalCandidateProvider ports. It rejects the assumption that any provider's extracted fact, persona, graph edge, summary, or context block is canonical truth. Such outputs enter ELL as untrusted candidates and remain subject to evidence, workspace, permission, version, deletion, and commit policy.

These projects are comparatively mature as deployable software, but their public evaluations use different models, prompts, budgets, datasets, and product configurations. Vendor- or project-reported latency, token, and accuracy results therefore motivate reproduction; they are not evidence that one substrate should be adopted as ELL's governing architecture. The present ELL proposal has not integrated or validated any of these systems end to end.

3.2 Empirical cautions for ELL

Figure 3.2: **Complexity is not evidence of learning.** Prior work motivates matched comparisons because richer architecture can add retrieval competition, negative transfer, and cross-domain fragility.

Recent evidence argues against assuming that more elaborate memory architecture implies better learning. A 2026 survey places cross-trajectory abstraction at the frontier of a broader storage–reflection–experience progression, making abstraction a field direction rather than an ELL-specific

novelty (Luo et al. 2026). Sequential-task experiments further show that external memory relocates rather than removes the continual-learning problem: abstract procedures can improve forward transfer while retrieval competition and negative transfer harm hard cases (Hu, Long, and Wang 2026).

Figure 3.3: **The field-wide progression.** Storage, reflection, and abstraction are established directions. ELL's contribution claim begins only with governed evidence, scope, revision, outcomes, and a successful behavioural comparison.

Cross-scenario comparisons report that a self-managed flat-file agent harness can rank above more specialised memory systems, which challenges the assumption that a fixed structured store will generalise across dialogue, code, search, and long-horizon tasks (Chen et al. 2026). Controlled analysis of graph and non-graph dialogue memory likewise finds that implementation details and strong simple baselines can matter as much as the graph abstraction itself (S. Hu et al. 2026). These findings require ELL to earn every layer through ablation. Graph association, learned memory operations, and specialised indexes remain optional conditions until they beat a simple baseline under the same evidence and cost budget.

Trustworthiness is also a retrieval problem, not only a write problem. Semantically related memory can be inappropriate for the current task and can create cross-domain leakage, sycophancy, tool drift, or memory-induced prompt attacks (J. Zhang et al. 2026). ELL therefore treats retrieval as a policy-bound admission decision and measures both relevant inclusion and harmful inclusion.

3.3 Reflection, reasoning memory, and non-parametric learning

Generative Agents introduced periodic reflection over accumulated observations, producing higher-level statements that could themselves be retrieved (Park et al. 2023). Reflexion showed that verbal feedback stored in episodic memory can improve repeated task attempts without weight updates (Shinn et al. 2023). ExpeL extended this idea by extracting insights across training experiences and transferring them to test tasks (Zhao et al. 2024). ReasoningBank distils strategies from both successful and failed trajectories and combines them with memory-aware test-time scaling (Ouyang et al. 2026). Reflective Memory Management used forward-looking summarisation and backward-looking evidence-based retrieval refinement (Z. Tan et al. 2025).

These systems establish reflection as a useful learning operator. ELL adopts that operator but makes a stricter distinction: a reflection is a candidate interpretation, not yet a trusted concept. This distinction enables explicit evidence thresholds, counterevidence checks, and lifecycle transitions. Self-judged success or failure is useful as one outcome signal but cannot be the sole authority for canonical learning; independent validators, task rewards, user corrections, and uncertainty must be retained where available.

3.4 Semantic, procedural, and graph-based abstraction

A second line of work converts experience into structured knowledge or reusable procedures. Voyager stores executable skills; Agent Workflow Memory induces reusable task routines; and A-MEM dynamically organises memory notes through contextual attributes and links (G. Wang et al. 2023; Z. Z. Wang et al. 2025; Xu et al. 2025). Temporal Semantic Memory aggregates temporally continuous evidence into durative user representations (Su et al. 2026).

The closest recent systems move explicitly from episodes to abstractions. DCPM separates fast fact updates from slower induction of schemas, intentions, and cross-domain patterns, with evidence links to supporting facts (Fei et al. 2026). GAAMA constructs a typed graph with episode, fact, reflection, and concept nodes (Paul, Sharma, and Sareen 2026). PlugMem derives semantic and procedural knowledge graphs from standardised episodic traces and maintains provenance to source episodes (Yang et al. 2026).

ELL is complementary to these architectures rather than a claim to precede them. Its intended distinction is the epistemic lifecycle of a concept. The core research object is not only a graph node or summary, but a claim with:

- separate supporting and contradicting evidence;
- an explicit applicability scope;
- a confidence calculation based on observable signals;
- temporal validity and version lineage;
- recorded downstream applications and outcomes;
- direct concept-level evaluation.

The implementation can use a graph, relational database, vector index, or a combination. The contribution is the contract and lifecycle, not a requirement for one storage technology.

3.5 Learned memory-operation policies

AgeMem exposes store, retrieve, update, summarise, and discard as tool actions and learns a unified short- and long-term memory policy with progressive reinforcement learning (Y. Yu et al. 2026). AtomMem decomposes management into atomic create, read, update, and delete operations and learns their orchestration with supervised and reinforcement learning (Huo et al. 2026). MemSkill instead represents extraction, consolidation, and pruning as reusable skills selected by a controller and revised by a designer that examines hard cases (H. Zhang et al.

2026).

ELL adopts the separation between atomic operations and policy, adaptive operation selection, and explicit learning from policy failures. These map to versioned MemoryOperationPolicy, ExtractionPolicy, ConsolidationPolicy, RetrievalPolicy, and ForgettingPolicy ports. A fixed heuristic and a learned policy should be interchangeable under the same typed inputs, outputs, budgets, and audit traces.

The rejected assumption is that task reward alone grants authority to mutate durable memory. Reward design can favour short-term benchmark success, opaque policies can make a write difficult to explain, and update or delete actions can destroy evidence needed for correction and scientific audit. ELL therefore permits learned policies to propose operations, but deterministic services enforce append-only provenance, authorization, idempotency, workspace isolation, tombstones, and canonical commit. These methods are recent research systems evaluated on bounded benchmark suites; their transfer, reward robustness, and deletion safety remain experimental questions.

3.6 Self-scaffolding and learned orchestration

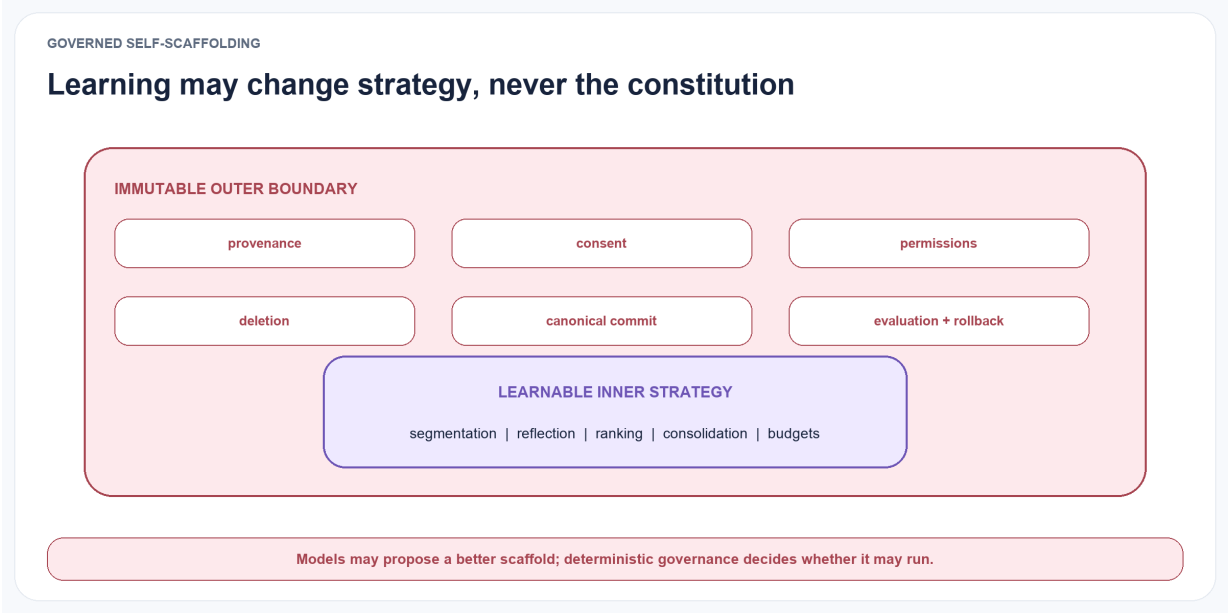

Figure 3.4: **Learning strategy remains inside an immutable constitution.** Segmentation, reflection, ranking, consolidation, and budgets may be learned; provenance, consent, permissions, deletion, canonical commits, evaluation, and rollback may not.

Ornith-1.0 extends policy learning from task solutions to the harness that produces them. At each reinforcement-learning step, the model first refines a task-specific scaffold from the task and its previous scaffold, then generates solution rollouts conditioned on that scaffold. Rollout reward is propagated to both stages, allowing per-category orchestration strategies to mutate and be selected together with the solutions they induce (Ornith Team 2026).

Its trust design is as relevant to ELL as its optimisation objective. Ornith keeps the environment, tool surface, and test isolation outside the learned scaffold; a deterministic monitor assigns zero reward to specified boundary violations, while a frozen language-model judge serves as a veto for intent-level gaming rather than as the primary reward. Its asynchronous training also discounts stale off-policy tokens so that old trajectories have diminishing influence on a newer policy.

ELL adopts the scaffold as a learnable object but changes its authority and deployment setting. Ornith describes self-scaffolding during model post-training for agentic coding; it does not by itself establish safe longitudinal self-modification over private personal experience. ELL therefore externalises the scaffold as a typed, versioned policy object and learns it in a slower governed loop. Candidate scaffolds may alter reflection scheduling, evidence selection, consolidation, retrieval, budgets, retries, and abstention. They may not alter canonical evidence, consent, permissions, deletion, provider-egress rules, evaluator isolation, or commit authority. Their outcomes

are tested through replay, time-forward evaluation, shadow mode, canaries, and rollback before promotion.

The cited Ornith source is a technical release note rather than a peer-reviewed methods paper. ELL therefore treats its self-scaffolding mechanism and reported safeguards as design hypotheses to reproduce and ablate, not as established evidence that the approach will transfer to experience learning.

3.7 Experience graphs and evolving skills

Experience-oriented systems increasingly represent reusable behaviour rather than only conversational facts.

EXG organises successful and failed trajectories into an experience graph for online growth and offline reuse (Jin et al. 2026). ReasoningBank and ExpeL distil general strategies across trajectories (Ouyang et al. 2026; Zhao et al. 2024). MUSE-Autoskill manages skill creation, storage, selection, evaluation, refinement, and skill-level experience under one lifecycle (Lin et al. 2026). Memento-Skills uses a learned router and reflective read-write loop to evolve structured external skills (Zhou et al. 2026).

ELL adopts graph-organised candidate generation, contrast between successes and failures, explicit skill lifecycle, and cross-task transfer measurement. A proposed SkillVersion should contain immutable identity, preconditions, scope, implementation or instruction content, lineage, tests, approval requirements, and rollback metadata. Each use should emit an ApplicationReceipt linking the exact skill and concept versions, retrieved evidence, action, cost, validator, and independently observed outcome. Outcome history may propose a new SkillVersion; it must not silently rewrite the version that was executed.

The rejected assumptions are that a skill file is self-validating, that repeated reuse proves causality, or that an LLM's reflection is an independent outcome measure. The named systems offer promising evidence in web, software-engineering, reasoning, and skill benchmarks, but do not yet establish universal transfer or safe autonomous modification. ELL therefore places ExperienceGraph, SkillGenerator, SkillRouter, and SkillEvaluator behind experiment ports and compares them under fixed token, latency, and validation budgets.

3.8 Neural and parametric test-time memory

Titans introduces a neural long-term memory module that learns to compress historical context and combines it with attention (Behrouz, Zhong, and Mirrokni 2025). Nested Learning interprets models and optimizers as nested optimization problems with distinct context flows; its HOPE architecture combines a self-modifying sequence model with a continuum memory system (Behrouz et al. 2025). TMEM distils experience into online LoRA updates so that a fast parameter state changes subsequent behaviour within an episode (Ren et al. 2026).

These approaches suggest promising accelerators for long context, fast adaptation, and policy specialization.

ELL may evaluate them through NeuralMemoryAccelerator or ParametricAdaptation ports while separately recording the source data, training objective, base model, update sequence, parameter delta, evaluation result, and deletion obligations. They are not suitable as the sole canonical store: a parameter update does not natively provide statement-level provenance, deterministic

correction, workspace isolation, selective export, or reliable evidence-aware deletion. Their current evidence is primarily model- and benchmark-level; reproducibility, catastrophic interference, privacy erasure, and auditability remain release gates.

3.9 Evaluation of memory and intuition

LongMemEval evaluates information extraction, cross-session reasoning, temporal reasoning, knowledge updates, and abstention in long-term interactive memory (Wu et al. 2025). MemoryAgentBench adds incremental multi-turn evaluation of accurate retrieval, test-time learning, long-range understanding, and selective forgetting (Hu, Wang, and McAuley 2025). MemBench broadens factual and reflective memory evaluation across interaction roles (H. Tan et al. 2025), while MemoryArena couples memory with later action in multi-session environments (He et al. 2026).

LoCoMo-Plus is especially relevant to the product-facing intuition hypothesis because it tests cue–trigger semantic disconnect: a later situation requires an implicit constraint from an earlier, lexically distant interaction (Li et al. 2026). Mem2ActBench tests whether prior preferences and task state are proactively applied to tool selection and parameters rather than merely recalled in an answer (Shen et al. 2026). Neither benchmark directly labels ELL-style concepts, evidence sets, or revision lineage, but both provide external tests of whether the proposed layer changes behaviour.

ELL will use these benchmarks for comparability, but none alone establishes the intended intuition capability.

An ELL-specific sealed benchmark must additionally measure evidence-supported compression, structurally distant transfer, calibrated uncertainty, change-point detection, correction latency, version and deletion lineage, and whether applying a retrieved concept improved an independently measured outcome. Product-facing shorthand and proactive relevance remain secondary external-validity tests. Results must include background consolidation cost and compare full ELL-Core with simple fixed baselines before any learned, graph, external-memory, or neural extension is considered.

3.10 Earlier foundations: revision, time, and provenance

ELL's governance mechanisms also have important predecessors outside the recent LLM-memory literature. Truth Maintenance Systems recorded justifications for beliefs and revised them when assumptions changed (Doyle 1979); Assumption-based Truth Maintenance Systems maintained alternative environments under inconsistent information (de Kleer 1986). AGM belief revision formalised contraction and revision of logically closed belief sets under new information (Alchourrón, Gärdenfors, and Makinson 1985). Bitemporal database work distinguished when a statement is valid in the world from when the database learned or recorded it (Clifford and Isakowitz 1992). W3C PROV defines interoperable entities, activities, agents, and qualified derivation relations for provenance exchange (Lebo, Sahoo, and McGuinness 2013).

ELL does not reproduce the formal guarantees of these traditions, and natural-language concepts are not deductively closed belief sets. The relevant lesson is narrower: reasons, assumptions, valid time, recorded time, derivations, and retraction dependencies should be explicit rather than buried inside generated prose. ELL's research contribution can only be the operational combination and empirical evaluation of these ideas for language-agent experience streams.

3.11 Positioning summary

Family	Representative systems	Role in the ELL study
Context and production memory	Letta/MemGPT, Mem0, Zep/Graphiti, Hindsight, TencentDB Agent Memory	Strong external systems to reproduce selectively; never assumed to be canonical ELL state
Experience abstraction	Reflexion, ExpeL, RMM, DCPM, GAAMA, PlugMem	Nearest methodological comparators for reflection, insights, concepts, and semantic or procedural abstraction
Agent-controlled memory	AgeMem, AutoMEM, MemCon (Y. Yu et al. 2026; Chen et al. 2026; Jiang et al. 2026)	Exploratory policy comparators after deterministic ELL-Core is evaluated
Retrieval infrastructure	exact vector search, HNSW, TurboVec	Replaceable candidate-generation substrates; they do not define learning semantics
Revision and provenance	TMS, ATMS, AGM, bitemporal databases, W3C PROV	Earlier foundations that constrain ELL's novelty claim and inform its audit contract
ELL-Core	evidence, episodes, reflections, concepts, applications, outcomes	The object of the confirmatory test: whether governed concepts improve transfer without violating guardrails

3.12 Cognitive inspiration and its limits

The episodic-semantic distinction and complementary learning systems provide useful engineering metaphors (Tulving 1972; McClelland, McNaughton, and O'Reilly 1995; Kumaran, Hassabis, and McClelland 2016).

They suggest preserving specific experiences while separately integrating common structure. However, ELL is not a neuroscientific model. LLM-generated natural-language concepts, database records, and scheduled consolidation jobs are functional approximations. Biological terminology is used to clarify roles, not to claim mechanistic equivalence.

4 Experience Learning Layer Architecture

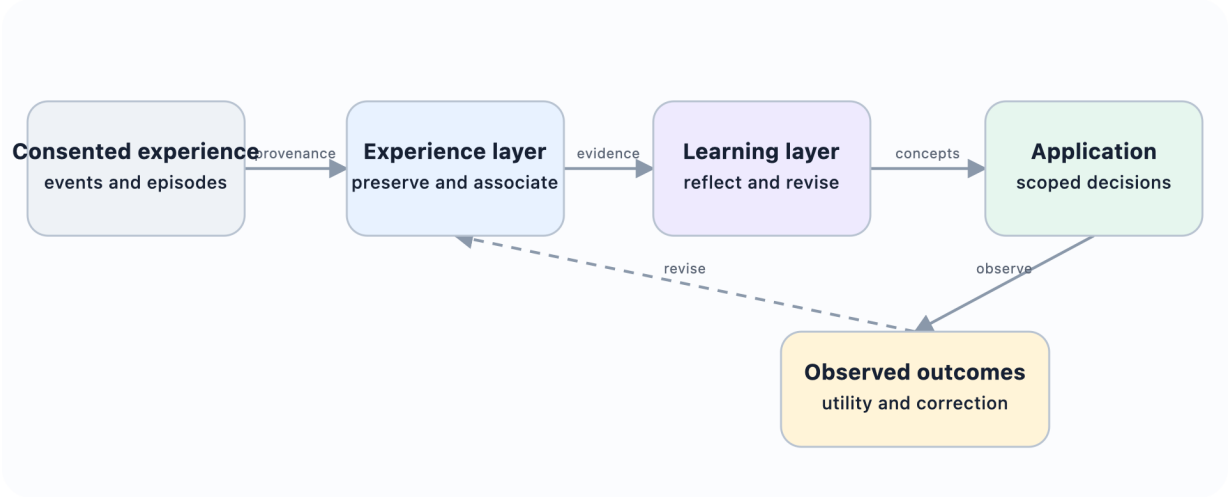

Figure 4.1: **Where the Experience Learning Layer sits.** ELL turns consented experience into evidence-backed, revisable concepts, then uses observed outcomes to improve or challenge them.

ELL is organised as a write–associate–reflect–consolidate–apply–evaluate loop. Each component exposes a narrow interface so that models, stores, and retrieval methods can be replaced independently.

The architecture separates three concerns. The input and capture layer normalises consented chats, recordings, documents, tool traces, and connectors into immutable sources, events, and episodes. Replaceable memory and retrieval infrastructure stores or indexes projections for association and candidate generation. The canonical learning layer governs hypotheses, concepts, applications, outcomes, versioning, permissions, and deletion.

External memory services and indexes can propose candidates, but they do not define canonical truth, identity, policy, or learning state.

4.1 ELL-Core and deferred extensions

The confirmatory system is deliberately small. ELL-Core contains seven durable objects: `SourceArtifact`, `Episode`, `Reflection`, `ConceptVersion`, `EvidenceLink`, `ApplicationReceipt`, and `Outcome`. `AuditEvent` records lifecycle operations. Associations may be computed ephemerally or stored as disposable projections. Skills, graphs, learned memory-operation policies, provider-specific memory objects, neural state, multimodal capture, and product clients are deferred extensions.

This boundary is methodological, not only architectural. If a concept layer cannot beat simple retrieval and direct-insight baselines with this minimum object set, adding a graph database,

learned controller, or specialised vector index would make attribution harder rather than rescue the claim.

4.2 Canonical episode store

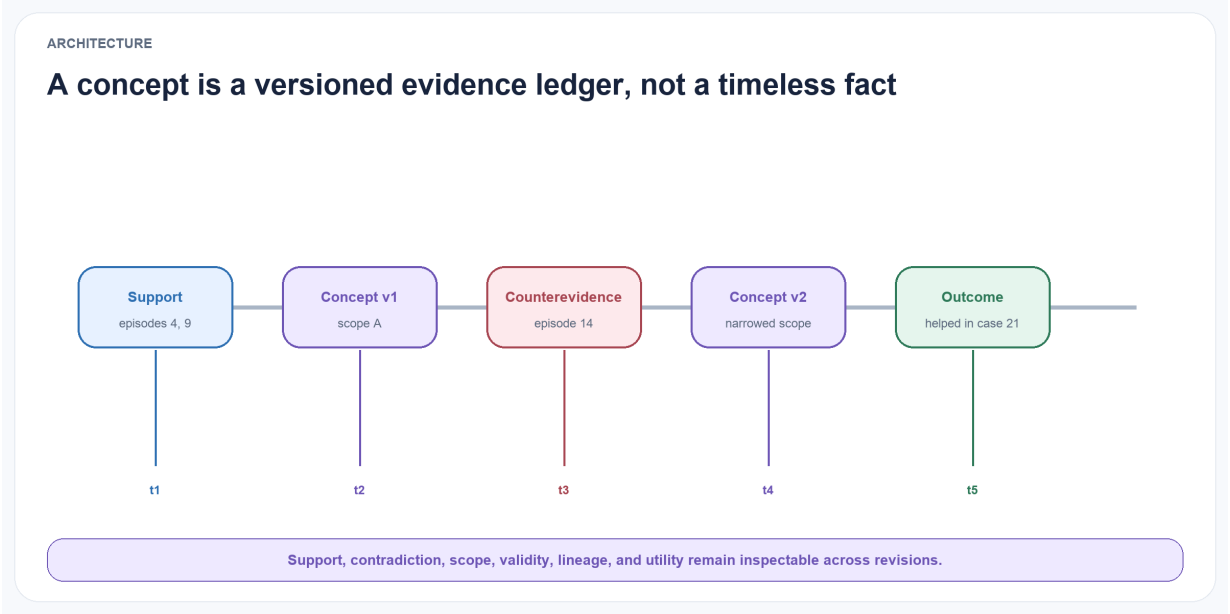

Figure 4.2: **A concept is a versioned ledger rather than a timeless fact.** Supporting episodes, counterevidence, revisions, and observed utility remain connected across valid and transaction time.

The input is a canonical episode rather than an application-specific chat message. An episode is represented as e_i=(id_i,t_event_i,t_observed_i,x_i,o_i,a_i,y_i,s_i,p_i), where id_i is the episode identifier, t_event_i and t_observed_i are event and observation time, x_i is context, o_i is an observation, a_i is an action, yi is an outcome, si is source metadata, and pi contains privacy and access-control metadata. The two timestamps allow delayed reports and later corrections to be represented.

The raw episode store is append-only. Corrections create new records linked by relations such as corrects, supersedes, or retracts. This preserves an audit trail and prevents an abstraction process from silently rewriting its own evidence.

4.3 Association layer

The optional association layer creates typed links between episodes. Initial relation types are:

- semantic similarity;
- shared entities or participants;
- temporal proximity or sequence;
- shared goal or task;
- shared action pattern;
- similar outcome;
- candidate causal relation;
- contradiction or correction.

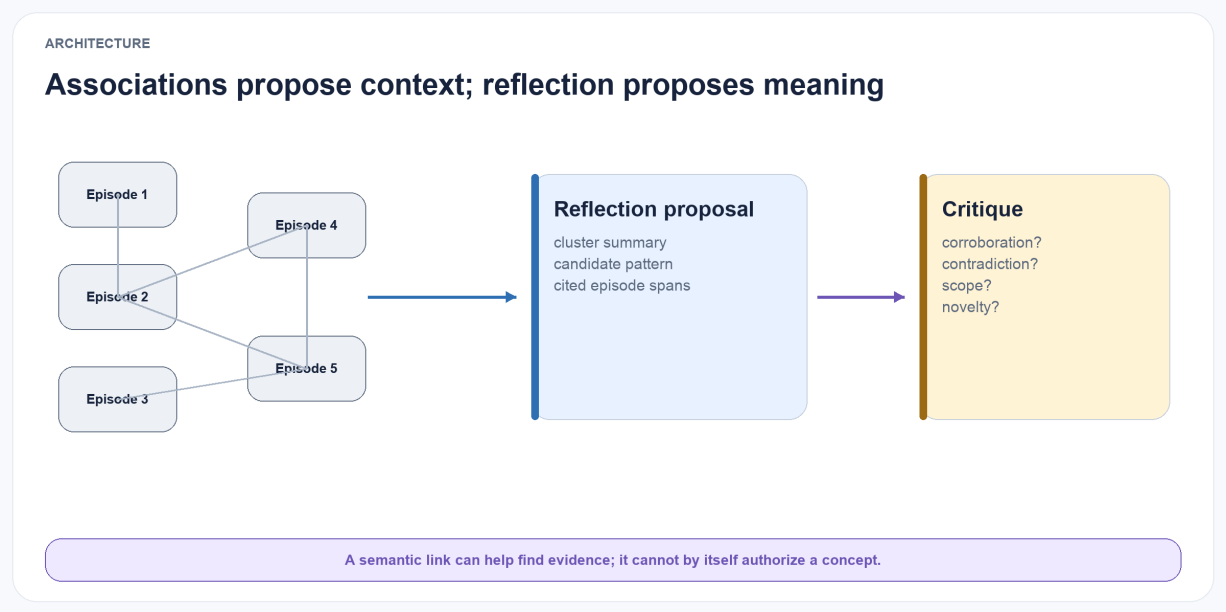

Figure 4.3: **Association and reflection have different authority.** Links and clusters help select context; a bounded reflection proposes meaning; critique checks support, contradiction, scope, and novelty before consolidation.

Only some edges are objective. Temporal sequence and shared identifiers can be deterministic; causal and contradiction edges may be model-generated hypotheses. Every edge therefore records its method, score, model or rule version, and source evidence.

The layer supports both local neighbourhood retrieval and clustering. Reflection should not operate only on the nearest embeddings because lexically distant episodes may share a goal, failure pattern, or outcome. Hybrid candidate generation combines semantic, structural, temporal, and outcome-based signals. The first benchmark also includes lexical-only, vector-only, and deterministic gold-cluster conditions so the value of association is measured rather than presumed.

4.4 Reflection Engine

A reflection is a provisional interpretation: r_j=(h_j,t_j,s_j,p_j,n_j,u_j,z_j).

Here hj is a claim or question, tj is reflection type, sj is scope, pj and nj are supporting and counterevidence episode sets, uj is uncertainty metadata, and zj is review status.

Reflection types include:

- observation or recurring pattern;
- causal hypothesis;
- strategy or procedural lesson;
- preference hypothesis;
- anomaly or exception;
- unresolved question;
- candidate correction to an existing concept.

The Reflection Engine is triggered by configurable events: a cluster reaching a minimum size, a repeated failure, a surprising outcome, new contradiction, elapsed time, or an explicit request. It produces structured output and is followed by an evidence-validation pass. Reflections may remain unverified, be marked supported, or be rejected before they reach the Concept Engine.

4.5 Concept Engine

A concept is a reusable, versioned proposition or strategy: c_k^(v)=(q_k,s_k,a_k,i_k,p_k,n_k,g_k,t_k,v,z_k

Here qk is the proposition, sk is scope, ak is a set of applicability conditions, ik is the expected implication or recommended behaviour, pk and nk are supporting and counterevidence sets, gk is confidence, tk is temporal validity, v is version, and zk is lifecycle state.

Concept types include descriptive patterns, user preferences, causal hypotheses, strategies, constraints, and procedural rules. An initial implementation should focus on textual concepts, but the schema permits executable procedures or references to code artefacts.

The Concept Engine performs five operations:

1. Propose: create a candidate concept from compatible reflections.
2. Merge: combine concepts that express the same claim and scope.
3. Split: separate an over-broad concept when evidence supports distinct conditions.
4. Revise: create a new version with changed claim, scope, validity, or confidence.
5. Retire or supersede: stop active use while preserving lineage.

Promotion requires evidence from more than one episode unless a domain-specific policy explicitly permits single-source facts. Evidence diversity is measured across time, source, participant, task, and surface form to reduce duplicate episodes being mistaken for independent support.

4.6 Concept registry and evidence ledger

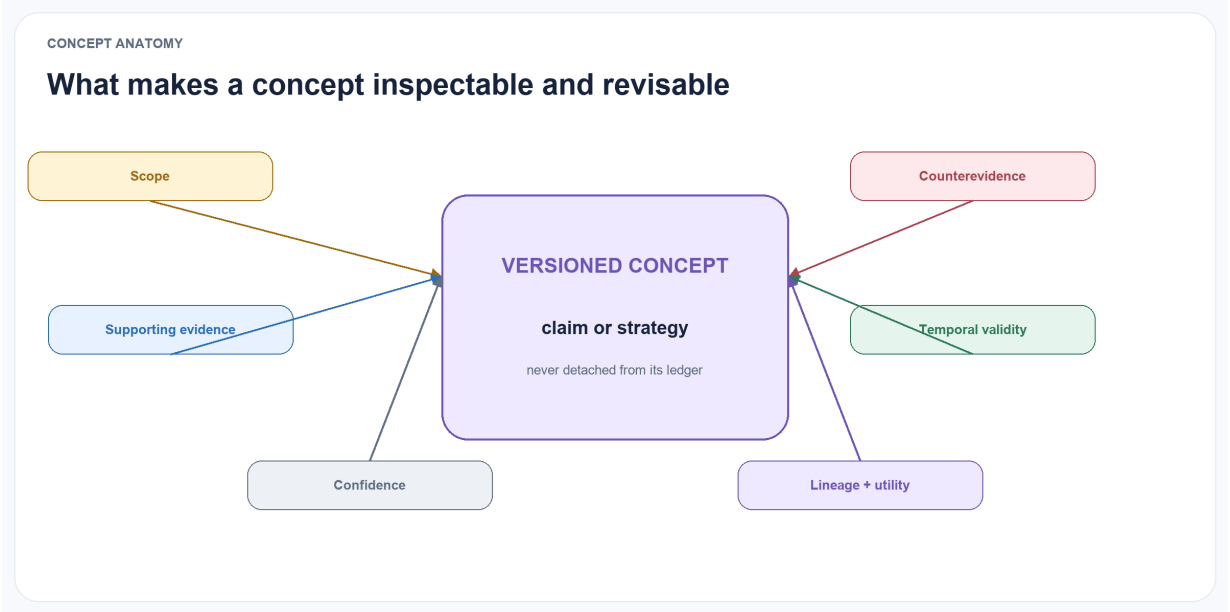

Figure 4.4: **Anatomy of a revisable concept.** Scope, supporting evidence, counterevidence, temporal validity, confidence, lineage, and utility remain first-class fields around the versioned claim or strategy.

The concept registry contains the current active version of each concept and its full version chain. The evidence ledger is append-only and records:

- which episodes supported or contradicted a reflection;
- which reflections contributed to a concept version;

- which model, prompt, rule, or human action produced a change;
- which concepts were retrieved for a decision;
- what action was taken and what outcome followed;
- deletions, access changes, and invalidations.

This ledger supports explanation and reproducibility. A user or evaluator should be able to ask: "Why does the system believe this?", "What evidence disagrees?", "When did this change?", and "Did using it help?"

4.7 Learning retrieval and application

At decision time, the retrieval service selects a mixture of concepts and source episodes. Concepts provide compact general guidance; episodes provide specificity, exception handling, and verification. The retrieval policy must be able to return either type independently.

A concept relevance score can initially be expressed as R(c,q)=w1 S1+w2 S2+w3 S3+w4 S4+w5 confidence(c)-w6 S6, where S1 through S4 represent semantic relevance, scope match, temporal validity, and observed utility; confidence(c) is concept confidence; and S6 is active contradiction. This formula is a design heuristic, not a validated scientific result. Weights will be tuned only on development data and frozen before test evaluation.

The retrieved package includes the concept statement, scope, confidence, current state, relevant support, relevant counterevidence, and source links. At the application boundary this compact, budgeted package is called an intuition packet. It may combine current concepts, selected episodes, procedures, open commitments, uncertainty, and known contradictions. It is a read object, not a new kind of memory. Applications can require a minimum confidence or allow contested concepts to be shown as uncertainty rather than instructions.

Formally, an intuition packet for query q, time t, policy context p, and budget b is I(q,t,p,b)=(C,*E*,A,*U*,X,*L*), where C* is the selected set of current concepts or hypotheses, E* is the minimum restored source evidence, A* contains applicable procedures or prospective commitments, U* is calibrated uncertainty, X* is material counterevidence or conflict, and L* is provenance and selection lineage. The packet is valid only if every item satisfies p, its estimated context cost does not exceed b, and its lineage resolves to permitted canonical records.

4.8 Outcome loop

Every use of a concept creates an application record: am=(xm,Cm,Em,dm,ym,um) .

Here xm is the task context, Cm and Em are the concepts and episodes used, dm is the decision, ym is the observed outcome, and um is utility. Outcome feedback can come from task reward, user correction, validator output, or later observed consequences. The system must not automatically treat every positive result as proof of causality; outcomes are recorded as validation signals with their own reliability. Repeated success can strengthen a concept, while failure can add counterevidence, narrow scope, or trigger revision.

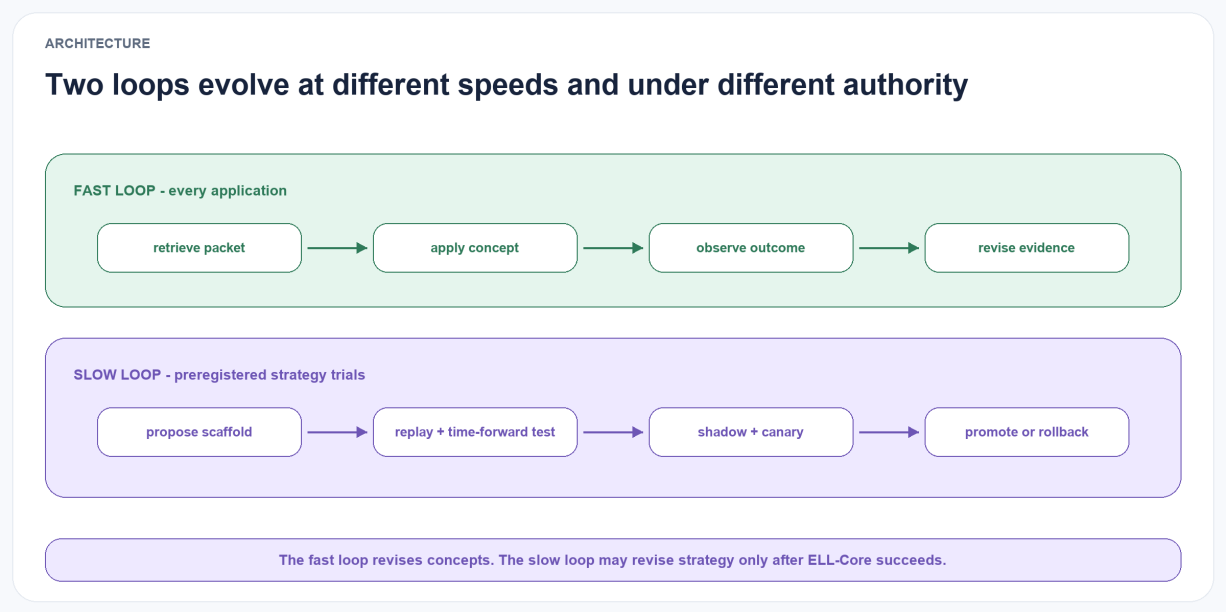

Figure 4.5: **Two loops, two speeds, two authorities.** The fast loop retrieves and revises concepts after applications. The slow loop evaluates learning strategies through replay, time-forward tests, shadow mode, canaries, promotion, and rollback.

4.9 Slow self-scaffolding loop

ELL separates learning about the world from learning how to perform that learning. The fast experience loop remains:

```
episode → reflection → concept → application → outcome → revision
```

The slower meta-learning loop is:

```
receipts and outcomes → scaffold proposal → replay → shadow trial → canary →
governed promotion or rollback
```

A `LearningScaffold` is a proposed extension object with an immutable identifier, version, parent, task category, model and prompt identifiers, available operation schemas, reflection triggers, evidence-selection policy, consolidation and retrieval parameters, resource budgets, error-recovery rules, abstention policy, validity interval, and lifecycle state. Each `ApplicationReceipt` in a self-scaffolding experiment records the exact scaffold version, selected evidence and concepts, model and tool versions, action, independent outcome, and total cost. This creates an auditable credit-assignment path from a strategy to the behaviour it induced.

The self-scaffolding controller is never a canonical writer. It proposes bounded changes to the inner learning strategy. The outer constitution remains deterministic and immutable to the controller: source provenance, consent, workspace isolation, purpose and sensitivity checks, schema validation, temporal consistency, deletion cascades, canonical commit rules, audit retention, tool permissions, provider egress, sealed evaluation, and rollback authority. A scaffold that violates any of these constraints is ineligible regardless of task reward.

This loop is a deferred extension rather than part of ELL-Core. Recording scaffold identity in experimental receipts can begin earlier, but automated proposal, selection, and promotion do not enter the first confirmatory study.

4.10 Lifecycle

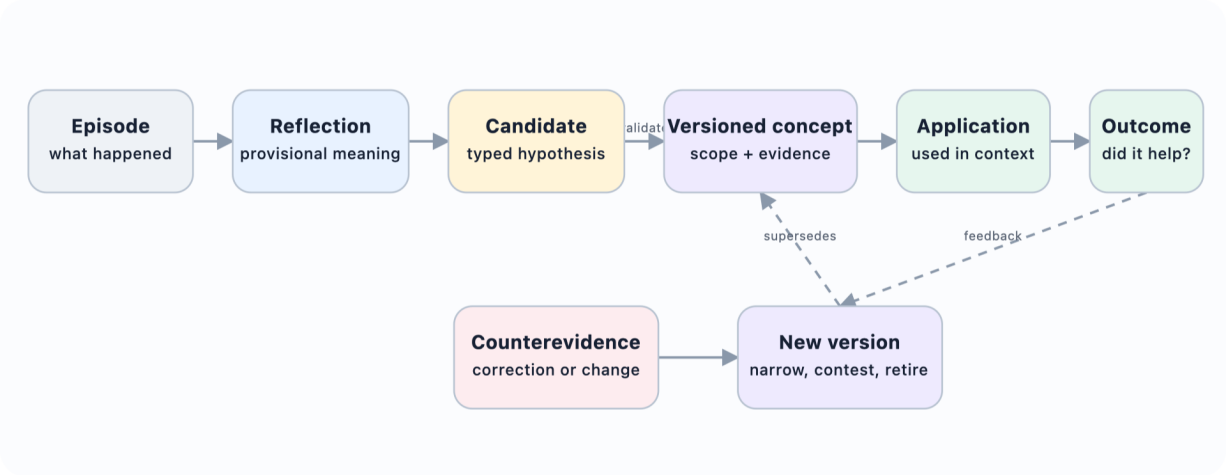

Figure 4.6: **From one episode to a revisable concept.** Interpretation remains provisional until evidence, scope, policy, and counterevidence support promotion into a versioned concept.

The lifecycle is intentionally reversible. Corroborated does not mean permanently true. Contested concepts may still be retrieved when their uncertainty is relevant. Superseded concepts remain available for historical questions, while retired concepts are excluded from ordinary guidance.

5 Proposed Algorithms

5.1 Minimum algorithm and comparison discipline

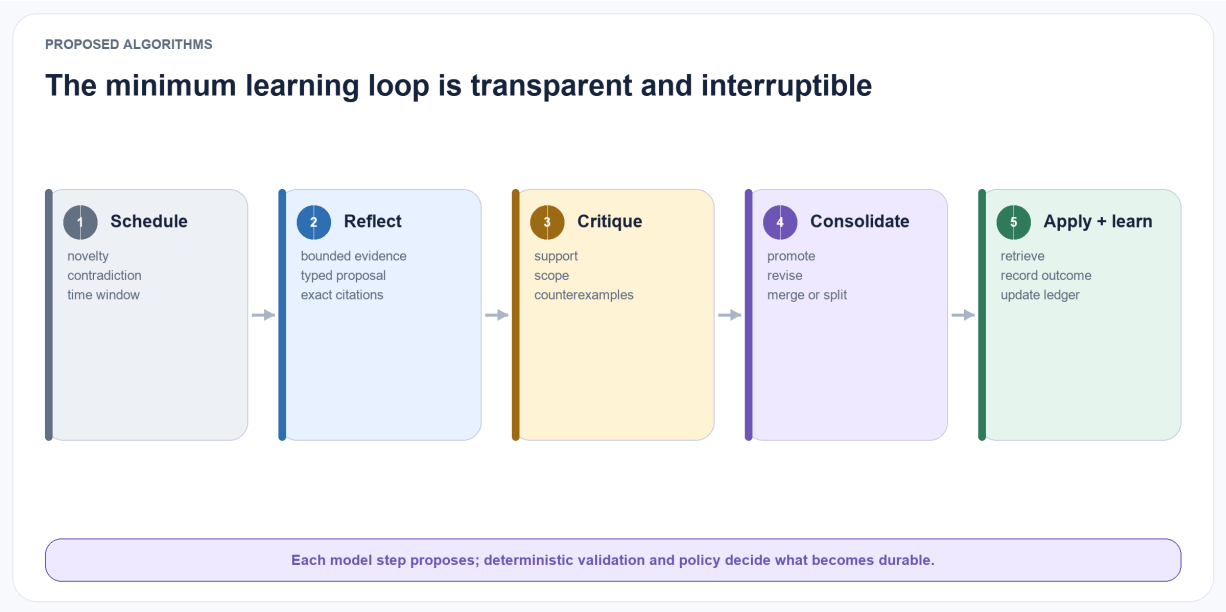

Figure 5.1: **The minimum learning loop.** Scheduling, bounded reflection, critique, consolidation, application, and outcome feedback remain transparent and interruptible.

The first implementation favours transparent rules over an optimised end-to-end pipeline. Each stage must expose its candidate inputs, accepted outputs, rejected outputs, cost, and reason code. A more complex policy is added only after the simpler condition has a frozen baseline result. This ordering separates the value of the concept contract from the value of a particular embedding model, graph, reranker, or learned controller.

5.2 Reflection scheduling

Running reflection after every episode would be expensive and may create redundant interpretations. ELL therefore uses event-based scheduling. An episode or cluster enters the reflection queue when one or more conditions are met:

- the cluster contains at least n distinct episodes;
- a failure or unusually high reward occurs;
- a new episode contradicts an active concept;
- association novelty exceeds a threshold;
- the concept has not been reviewed within a time window;

- a user or agent explicitly requests reflection.

The scheduler records why a job was triggered. This enables later analysis of whether a scheduling policy created useful concepts or unnecessary model calls.

5.3 Reflection generation and critique

The Reflection Engine receives a bounded evidence packet: selected episodes, existing nearby reflections, relevant active concepts, and an instruction to produce typed hypotheses. The generator must output structured fields rather than a free-form essay.

A separate validation pass performs four checks: 1. Entailment: Does each cited episode actually support the claim attributed to it?

2. Contradiction: Are there nearby episodes that disagree?
3. Scope: Is the claim broader than its evidence?
4. Novelty: Is the reflection meaningfully different from existing reflections or concepts?

The validator can be an independent model, a deterministic rule, a human reviewer, or a combination. Using a separate model call does not guarantee independence, so experiments will include same-model and cross-model validation conditions.

5.4 Concept proposal

Reflections are grouped by semantic compatibility and scope overlap. A proposed concept requires:

- at least m compatible reflections;
- at least d distinct supporting episodes;
- minimum evidence diversity;
- no unresolved high-confidence contradiction;
- an explicit scope and exception list;
- a concise proposition and expected implication.

The initial development policy will require support from at least three episodes across at least two distinct contexts. The threshold is not a claim about how many observations create knowledge. It is a preregistered control variable, swept only on development streams and frozen before the sealed test.

A simplified proposal procedure is: for each reflection cluster R: evidence = union(supporting episodes in R) counter = retrieve plausible counterevidence candidate = generate_concept(R, evidence, counter) checks = validate(candidate, evidence, counter) if checks pass promotion policy: register(candidate, state=“proposed”)

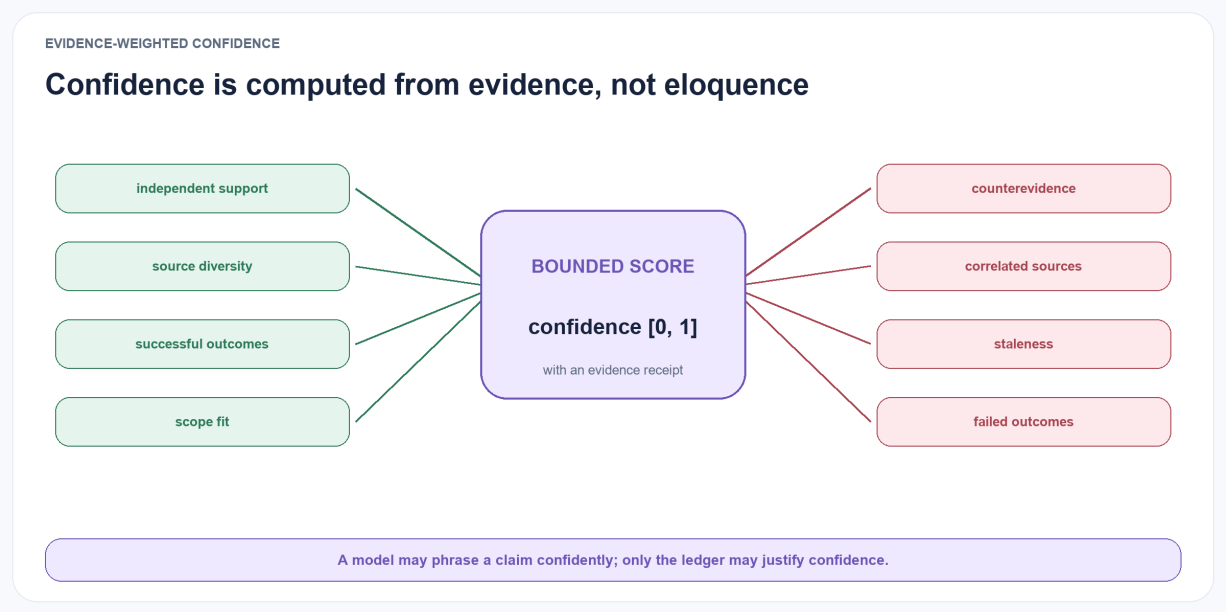

Figure 5.2: **Confidence comes from the ledger.** Independent support, diversity, outcomes, and scope fit raise a bounded score; counterevidence, correlation, staleness, and failed outcomes reduce it.

5.5 Evidence-weighted confidence

LLMs are not assumed to be calibrated judges of their own abstractions. ELL computes an operational confidence score from external features:

confidence(c)=1/(1+exp(-z_c)) , where zc is a weighted score that increases with supporting evidence, evidence diversity, and observed application utility, and decreases with counterevidence, age without revalidation, and validator disagreement.

The formula is deliberately transparent and will be calibrated on held-out development data. A fixed rule based on support, contradiction, and outcome counts is the mandatory baseline; logistic and Bayesian alternatives are secondary conditions. Confidence is never used to hide counterevidence, and it is not interpreted as a probability until calibration supports that interpretation. The raw counts and links remain visible.

5.6 Revision and temporal validity

When new evidence conflicts with a concept, the system chooses among four responses:

- add exception: the core claim remains valid but has a narrower boundary;
- narrow scope: the concept was over-generalised;
- change validity interval: the pattern was true during one period but is no longer current;
- replace claim: a new concept version better explains the evidence.

Revision produces a new immutable version linked by revises or supersedes. The old version retains its former validity interval and applications. This design supports historical reasoning and avoids treating changed preferences or conditions as contradictions that must erase the past.

5.7 Merge and split

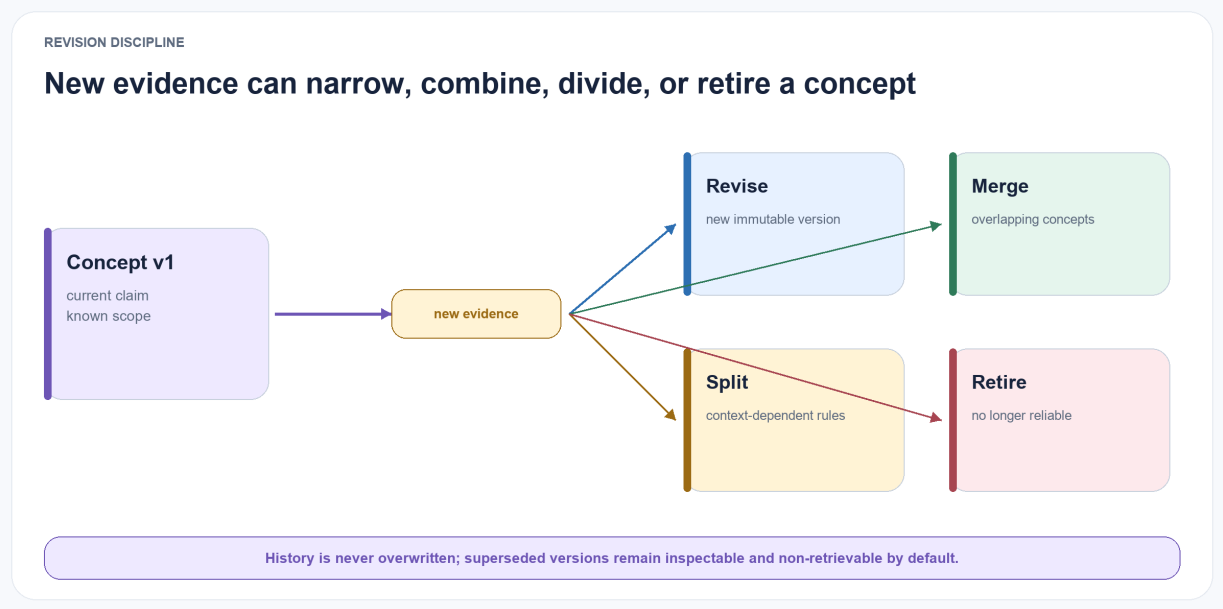

Figure 5.3: **Revision is branching, not overwriting.** New evidence may produce a new version, merge overlapping concepts, split context-dependent rules, or retire an unreliable concept.

Duplicate concepts increase retrieval noise, while over-broad concepts hide meaningful exceptions. Merge candidates are identified using proposition similarity, overlapping evidence, compatible scope, and similar implications. Split candidates are triggered when counterevidence clusters around a coherent condition or when application outcomes differ significantly by context.

Both operations require a lineage record. A merge creates a new concept that references all parents. A split creates child concepts whose evidence partitions are explicit. Automated operations above a risk threshold can require human approval.

5.8 Retrieval with evidence restoration

Concept retrieval follows two stages. The first stage selects concepts by semantic, structural, temporal, and utility signals. The second restores a small, query-specific set of source episodes. This prevents a compact concept from becoming an ungrounded instruction.

The retrieval budget is fixed in tokens or characters so that systems are compared under equal context cost.

This avoids rewarding a method merely for retrieving far more text. Results report both task utility and memory information density.

5.9 Deletion and invalidation

A user must be able to delete source data. Deletion cannot stop at the episode store because derived concepts may still encode the removed information. ELL maintains derivation links so a deletion request can:

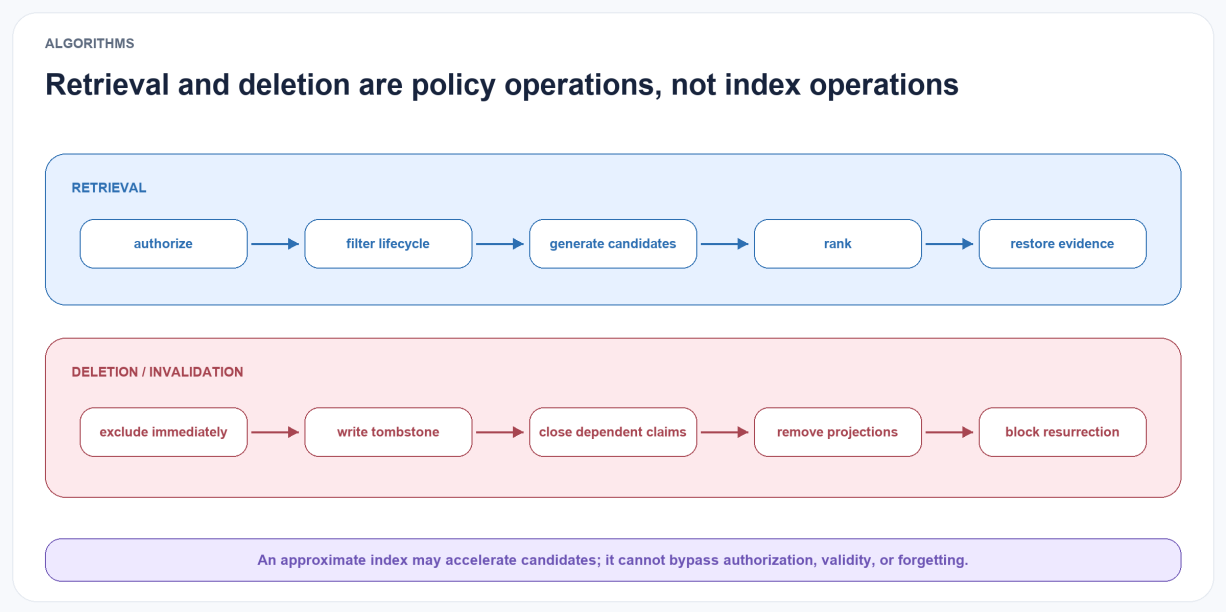

Figure 5.4: **Retrieval and deletion both enforce policy.** Authorization and lifecycle filtering precede ranking; deletion excludes immediately, writes a tombstone, closes dependent claims, removes projections, and blocks resurrection.

1. remove or tombstone the source episode according to policy;
2. remove it from support and counterevidence sets;
3. recompute affected confidence values;
4. contest, revise, or retire concepts that no longer meet support thresholds;
5. record the deletion cascade without retaining prohibited content.

This is both a privacy requirement and a scientific requirement: provenance is incomplete if derived claims cannot be invalidated when their evidence disappears.

5.10 Governed self-scaffolding extension

Ornith-1.0 suggests that the scaffold which elicits an agent's behaviour can itself be optimised jointly with task solutions (Ornith Team 2026). ELL adopts this as a second-order learning problem after the fixed-policy ELL-Core study. The initial implementation does not update model parameters. It searches over external, declarative `LearningScaffold` versions whose mutable fields are explicitly enumerated and schema-validated.

An initial scaffold trial proceeds as follows:

1. Select a task category and the current approved scaffold without exposing sealed outcomes.
2. Generate a small bounded set of typed mutations from the current scaffold and recorded development failures.
3. Reject candidates that change fields outside the sanctioned learning-policy surface.
4. Run multiple paired replays for each eligible candidate under identical sources, base model, tool surface, context budget, random seeds, and outcome opportunities.
5. Disqualify any trajectory that violates provenance, permission, deletion, evidence, tool, or evaluator-isolation invariants.
6. Compare the remaining candidates with the current scaffold on chronological development and out-of-time cases.

7. Run a selected candidate in shadow mode, then a category-limited canary with an explicit rollback target.
8. Promote it by creating a new immutable scaffold version, or retire it while preserving its proposal, trials, vetoes, costs, and rejection reasons.

The initial selection method is governed evolutionary search, not reinforcement learning: a model proposes several bounded mutations and deterministic experiment code selects among candidates after fixed gates. Contextual bandits or reinforcement learning become eligible only after application receipts provide sufficient independent, non-duplicated outcome evidence and a preregistered comparison justifies the additional opacity and cost.

Reward is evaluated only after hard governance gates. Eligible candidates are compared on time-forward task utility, structurally distant transfer, calibration, counterevidence and exception recall, correction latency, negative transfer, information density, and total write- plus read-path cost. Privacy leakage, cross-workspace access, unsupported sensitive inference, provenance loss, evaluator tampering, or incomplete deletion makes a trial ineligible rather than merely lowering a scalar score. A language-model judge may veto suspicious intent that deterministic rules cannot express, but it is frozen before evaluation and is never the sole reward source.

Policy staleness and evidence validity are kept separate. Outcomes collected under obsolete scaffold, model, prompt, or tool versions may be downweighted or excluded when estimating the value of a current scaffold. The underlying episodes are not weakened merely because they are old; their applicability is governed by valid time, observed time, correction, and explicit user evidence.

6 Data Model and Public Interfaces

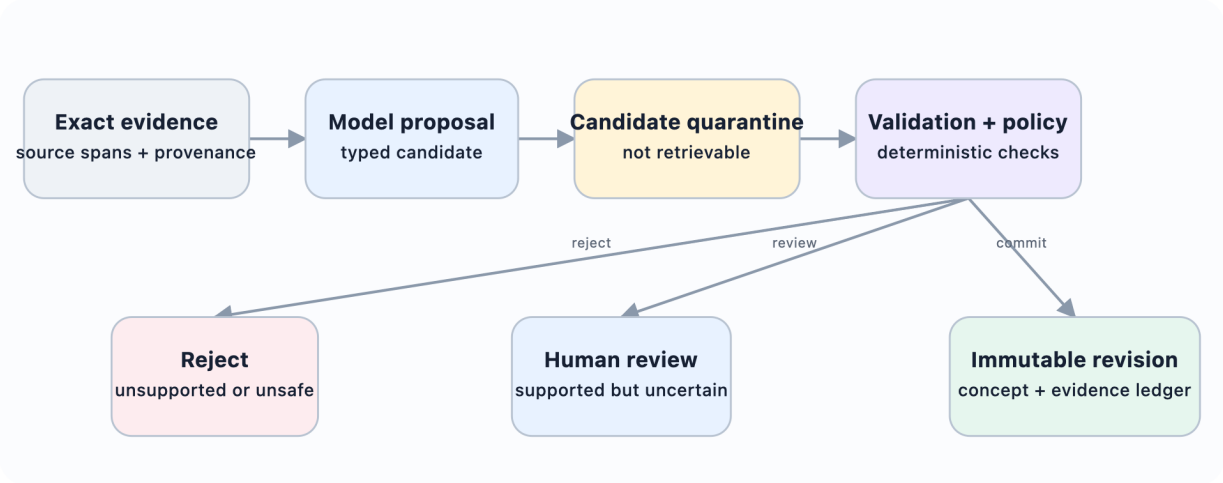

Figure 6.1: **Models interpret; deterministic code governs.** Generated interpretations enter quarantine. Schema validation and deterministic policy decide whether they are rejected, reviewed, or committed.

6.1 Core entities

Figure 6.2: **Six canonical objects carry the learning claim.** Source artifacts ground bounded episodes; reflections remain provisional; concepts are versioned; applications record use; outcomes independently record what happened next.

Entity	Purpose	Required fields
`SourceArtifact`	Immutable addressable evidence	ID, content hash, source type, stable spans, event and observed time, consent, sensitivity, workspace
`Episode`	Bounded experience derived from one or more sources	ID, context, observation, action, outcome, source spans, access metadata
`Reflection`	Provisional interpretation or question	statement, type, scope, support, counterevidence, uncertainty, review status
`ConceptVersion`	Reusable versioned claim or strategy	proposition, scope, conditions, implication, support, counterevidence, confidence, validity, version, lifecycle state
`EvidenceLink`	Typed derivation or challenge relation	source ID and span, target ID and version, relation, method, validator, timestamp
`ApplicationReceipt`	Immutable record of a concept's use	task, exact concept versions, restored evidence, decision, model, cost, timestamp
`Outcome`	Independently sourced evidence about an application	linked receipt, observation or reward, source, reliability, delay
`AuditEvent`	Content-minimised lifecycle record	actor, operation, object, prior version, new version, method, timestamp

`Association` is an optional typed projection used for candidate generation. `SkillVersion` and provider-specific memory assets are deferred extension objects and are not required to test the primary ELL-Core claim.

6.2 Concept states

- proposed: generated and valid enough to test, but not yet strongly supported;
- corroborated: meets evidence and validation thresholds;
- contested: important unresolved counterevidence exists;
- revised: a new immutable version has been created in response to changed evidence and is awaiting or recording validation;
- superseded: replaced by a newer or more precise concept;
- retired: no longer eligible for ordinary retrieval;
- deleted: content removed under deletion policy, with only non-sensitive structural audit metadata retained where legally permitted.

A revision never overwrites its parent. A revised version can later become corroborated, contested, superseded, retired, or deleted while its lineage remains traceable.

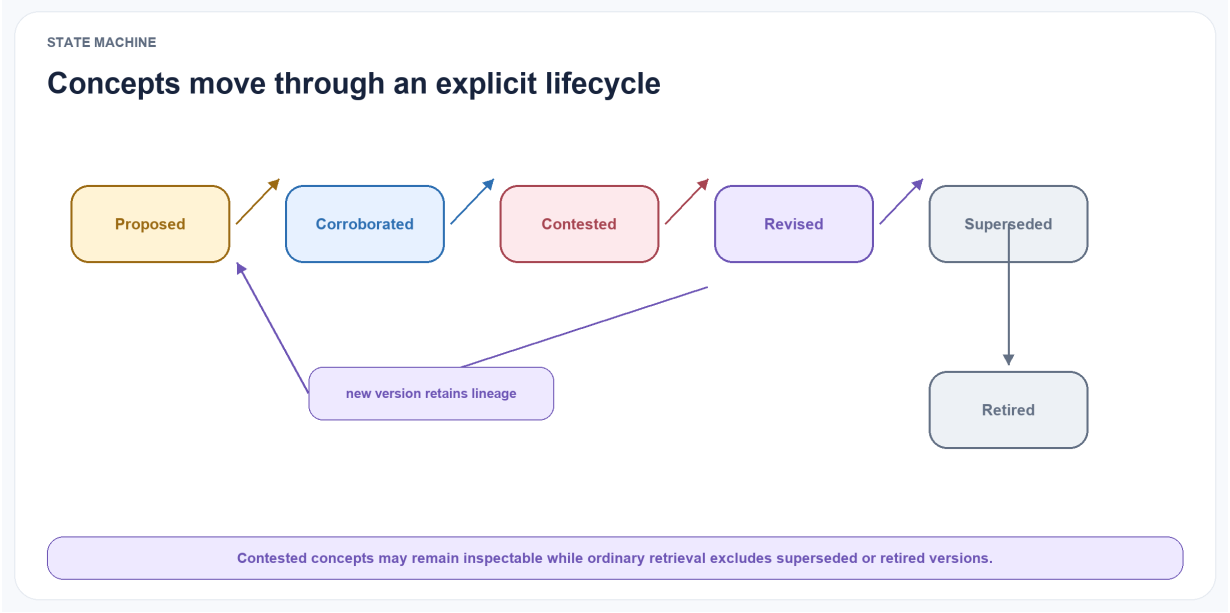

Figure 6.3: **The concept lifecycle is explicit.** Proposed concepts may become corroborated, contested, revised, superseded, or retired while immutable lineage preserves how the state changed.

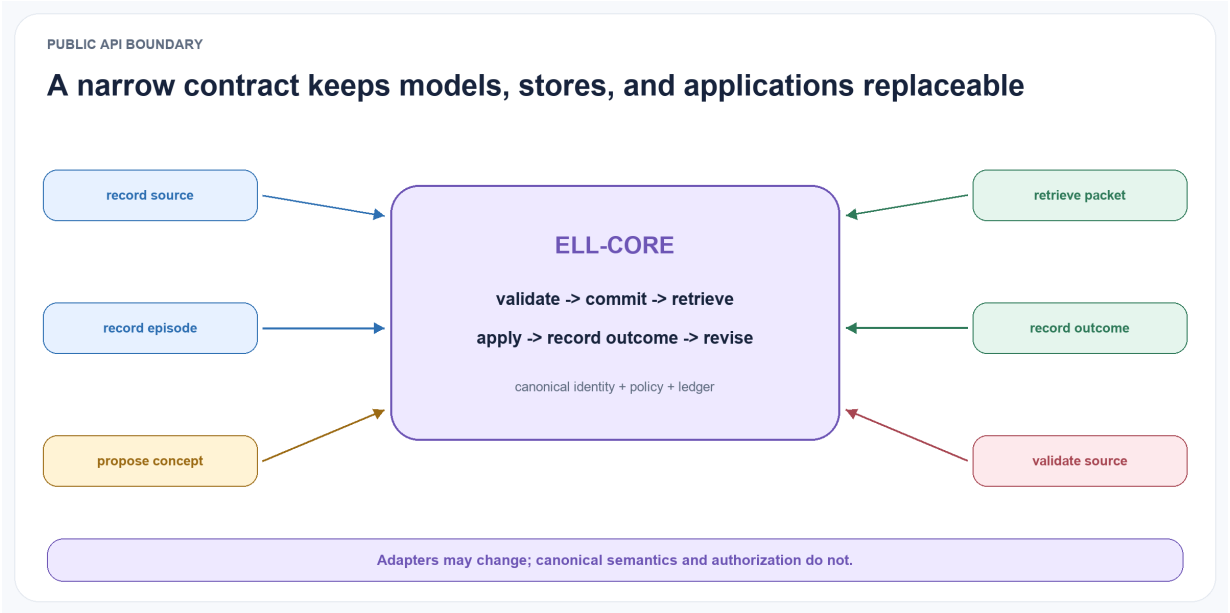

Figure 6.4: **A narrow API preserves replaceability.** Models, applications, stores, and retrieval systems use typed ports around ELL-Core while canonical identity, policy, lifecycle, and evidence stay inside the boundary.

6.3 API contract

A reference implementation should expose the following model-independent minimum operations:

```
record_source(source) -> SourceID
record_episode(episode) -> EpisodeID
propose_reflections(scope, trigger) -> list[Reflection]
reconcile_concepts(reflection_ids) -> list[ConceptVersion]
retrieve_learning(query, context, budget) -> LearningPacket
record_application(packet, decision) -> ApplicationID
record_outcome(application_id, outcome) -> list[ConceptUpdate]
explain_concept(concept_id, version=None) -> EvidenceReport
invalidate_source(source_id, policy) -> InvalidationReport
```

Association, skill proposal, graph traversal, provider import, and learned-policy APIs are optional ports layered around this contract.

LearningPacket is the central read object. It contains concepts, relevant evidence, conflicts, and budget accounting. It never contains only a naked concept statement.

6.4 Storage independence

The core schema is storage-neutral. A starter implementation uses an in-memory store and deterministic scan before adding SQLite and full-text search as the first persistent baseline. Exact vector search is evaluated before approximate indexes. Production adapters may later add PostgreSQL, TurboVec or another vector index, a graph database, or a hosted memory system. Storage choices are evaluated by correctness, write cost, query latency, deletion support, reproducibility, and operational simplicity rather than by architectural fashion.

The same principle applies to model providers. The engine depends on a structured-generation interface, not a vendor SDK. Reproducibility experiments use open-weight models, while optional adapters may support hosted models for comparison.

Association, extraction, consolidation, retrieval, and forgetting policies may become configurable or learned, but their outputs remain proposals behind stable typed ports. Deterministic code continues to enforce provenance, workspace isolation, permissions, deletion, schema validity, and canonical commit rules. Competing lexical, vector, graph, temporal, probabilistic, and model-driven policies remain interchangeable experimental conditions until evaluation establishes which combination is useful. A Zettelkasten-style linked-note representation may be included as a simple baseline, but static note linking is too limited to govern temporal change, uncertainty, counterevidence, outcome feedback, and user control.

7 Experimental Design

7.1 Study status

The evaluation is preregistered in this paper before results are collected. All thresholds, prompts, model versions, random seeds, exclusions, and statistical tests will be committed before the sealed test sets are run.

Any later changes will be reported as deviations rather than silently folded into the method.

7.2 Evaluation stages

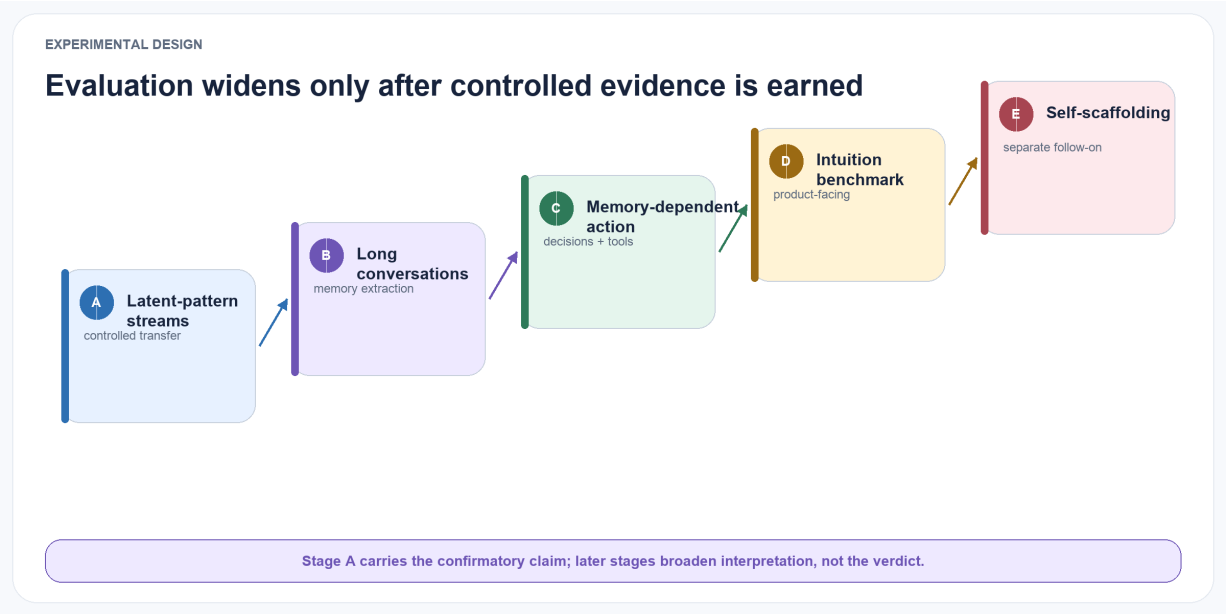

Figure 7.1: **Evaluation widens only after controlled evidence is earned.** Stage A carries the confirmatory transfer claim; Stages B-D broaden the task setting; Stage E is a separately gated self-scaffolding study.

7.2.1 Stage A: controlled latent-pattern streams

The project will release a synthetic benchmark in which each experience stream is generated from known latent concepts. A stream contains:

- recurring contextual rules;
- paraphrased surface forms;
- irrelevant but semantically similar episodes;
- exceptions with explicit conditions;
- contradictory observations;

- temporal change points;
- delayed outcomes;
- duplicated or correlated evidence.

Example latent concept: When a task depends on another team and the owner is not involved before commitment, delivery risk increases; the pattern does not apply to independently executable tasks.

The generator creates episodes that support, contradict, or fall outside the concept's scope. The benchmark therefore has gold labels for the concept, its conditions, its evidence, counterevidence, and validity interval.

Three stream lengths are planned: 50, 200, and 1,000 episodes. Difficulty increases through noise, lexical distance, exceptions, and concept drift. A sealed test generator seed prevents manual prompt tuning against the exact cases.

7.2.2 Stage B: long-term conversational memory

LongMemEval and LoCoMo will test whether ELL remains competitive on factual extraction, cross-session reasoning, temporal reasoning, updates, abstention, and long-range dialogue understanding (Wu et al. 2025;

Maharana et al. 2024). MemBench will add reflective-memory and efficiency measures where licensing and harness compatibility permit (H. Tan et al. 2025). MemoryAgentBench will test retrieval, test-time learning, long-range understanding, and selective forgetting in incremental multi-turn interaction (Hu, Wang, and McAuley 2025).

LoCoMo-Plus adds cue–trigger semantic disconnect, where a later prompt must activate an earlier latent constraint despite low lexical similarity (Li et al. 2026). It is the preferred external test for the structurally distant retrieval aspect of the ELL claim, although it does not provide gold concept lifecycles.

ELL is not expected to dominate span-recall tasks solely because it forms concepts. These benchmarks test whether the abstraction layer preserves ordinary memory performance and whether concept retrieval helps multi-session inference.

7.2.3 Stage C: memory-dependent action

MemoryArena will test whether information learned in earlier sessions improves later task execution (He et al.

2026). Mem2ActBench tests whether established preferences and task state are applied to tool selection and parameters rather than only restated in an answer (Shen et al. 2026). A smaller deterministic environment will be used for rapid development, while these external benchmarks test whether a result survives outside the synthetic generator.

7.2.4 Stage D: ELL intuition and outcome benchmark

The ELL-specific benchmark combines controlled organisational and personal-work streams with later tasks that require conversational shorthand, proactive but non-intrusive context, structurally distant transfer, adaptation at known change points, and explicit correction. Each decision records an ApplicationReceipt and an independently scored outcome. Gold data identifies required context, irrelevant private context, support, counterevidence, validity intervals, and deletion cascades. The benchmark therefore measures not only whether information was recalled, but whether a compact intuition packet selected the right evidence, improved behaviour, remained calibrated, and changed or forgot appropriately.

7.2.5 Stage E: governed self-scaffolding

After the ELL-Core verdict and external-validity gates are complete, a separate study tests whether ELL can improve the strategy by which it learns. Inspired by Ornith-1.0's joint optimisation of scaffolds and solution rollouts, this stage compares a frozen hand-specified scaffold, development-tuned fixed scaffolds, governed model-proposed scaffold mutations, and—only if receipts are sufficiently numerous—an adaptive controller (Ornith Team 2026). It does not modify the primary confirmatory result.

Every condition uses the same chronological sources, task-category labels, base model, tool surface, operation schemas, outcome opportunities, and total resource budget. Each scaffold receives multiple paired rollouts or deterministic replays. Development, out-of-time, shadow, and canary intervals are distinct; future outcomes, sealed labels, evaluators, and governance configuration are unavailable to the proposal policy. The experiment reports scaffold transfer across categories, improvement over the strongest frozen scaffold, promotion error, rollback success, policy stability, negative transfer, calibration, safety-veto rate, and total cost.

A deterministic monitor disqualifies specified violations. A separately frozen language-model judge may veto intent-level gaming but cannot grant reward or override a deterministic failure. Promotion requires improvement on the preregistered objective vector with no regression on consent, permission, provenance, deletion, evidence restoration, sensitive inference, cross-workspace isolation, or evaluator integrity.

7.3 Memory Dynamics and Intuition Simulation Lab

The evaluation will include a reproducible simulation laboratory for studying memory dynamics before any policy is considered for deployment. The laboratory replays deterministic, timestamped scenario streams through the same typed ports used by the reference architecture. It supports parameter sweeps and side-by-side policy comparisons for extraction, association, consolidation, retrieval, revision, and forgetting without granting an experimental policy authority to mutate canonical evidence. Each run fixes the scenario version, configuration, model and prompt identifiers, random seed, permissions, and resource budget. It emits immutable decision and ApplicationReceipts that resolve every selected context item, candidate mutation, committed version, action, outcome, evaluator judgement, and cost to its permitted source evidence.

Scenarios use synthetic or appropriately de-identified longitudinal personas representing both individuals and organisations. They include stable preferences, weak signals, sensitive attributes that must not be inferred or revealed, explicit consent changes, contradictions, temporal drift, delayed outcomes, workspace boundaries, and deletion requests. Chronological train, development, and sealed test intervals prevent policies from learning from future events; change points and

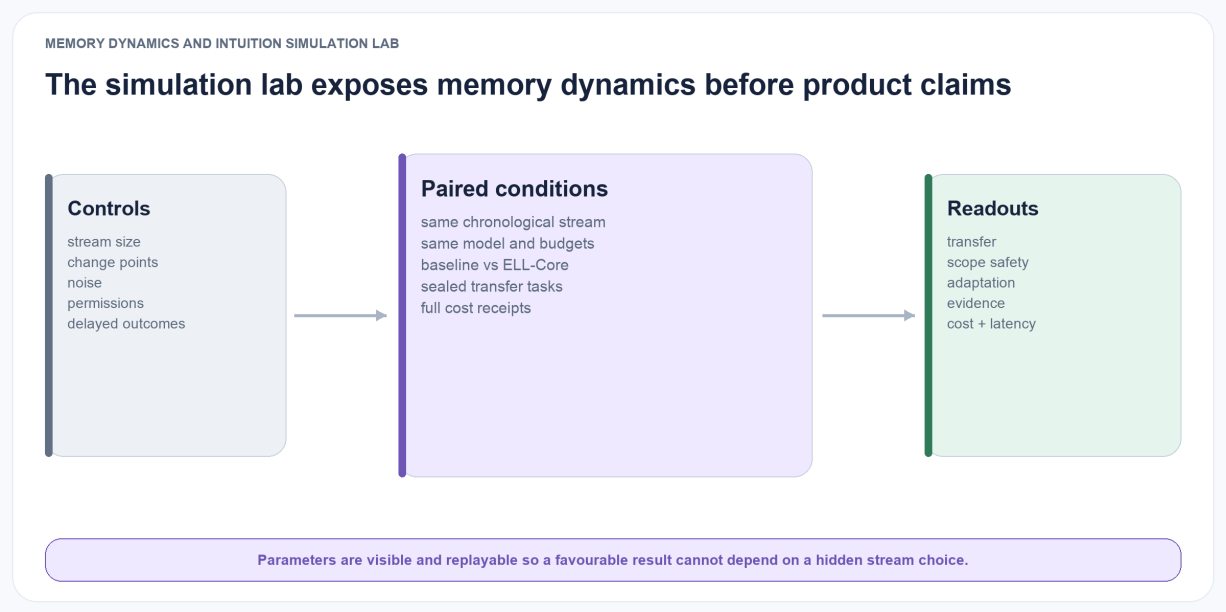

Figure 7.2: **The simulation lab makes hidden choices visible.** Stream size, noise, change points, permissions, and delayed outcomes feed paired baseline and ELL conditions with matched budgets and complete transfer, safety, adaptation, evidence, and cost readouts.

deletion events remain hidden from the system until they occur in replay. The laboratory compares no memory, raw-history or maximum-context prompting, ordinary retrieval, direct insight extraction, and ELL-Core ablations under matched sources, model conditions, context budgets, and outcome opportunities.

Preregistered primary measures cover retrieval precision and recall; faithfulness and calibration of claims; adaptation latency after change or correction; contradiction and stale-memory rates; transfer and downstream task utility; overpersonalisation and unjustified inference; privacy, consent, and cross-workspace leakage; deletion-cascade completeness; and latency, token, storage, and energy or hardware-time cost, including consolidation.

Scoring is deterministic wherever gold state and event traces permit. Outcome and safety judgements are produced by evaluators independent of the policy under test; subjective conversational naturalness is assessed separately by blinded human reviewers. Stochastic conditions use multiple preregistered seeds and report confidence intervals and effect sizes. This laboratory is an evaluation proposal, not evidence that the present implementation already provides adaptive intuition or safe learned memory operations.

For Stage E, the laboratory additionally stores `LearningScaffold`, `ScaffoldProposal`, `ScaffoldTrial`, and `ScaffoldDecision` artefacts. Each trial links its parent scaffold, bounded mutation, task category, policy and model versions, receipts, independent outcomes, resource use, hard-gate results, judge vetoes, and disposition. Old-policy outcomes may receive a preregistered staleness weight for policy credit assignment, but evidence validity continues to follow event and observation time rather than scaffold age.

7.4 Baselines

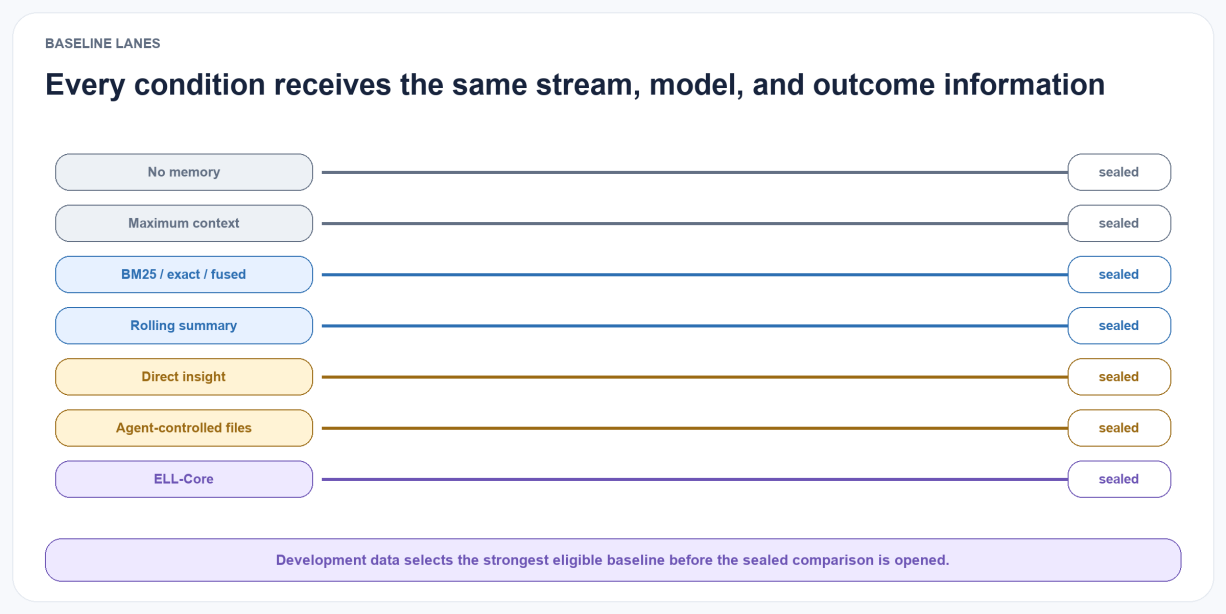

Figure 7.3: **Parallel baseline lanes.** No-memory, maximum-context, retrieval, summary, direct-insight, agent-controlled, and ELL-Core conditions receive the same chronological stream and model conditions before sealed evaluation.

At minimum, all experiments include:

1. no persistent memory;
2. full or maximum available context;
3. BM25 retrieval over raw episodes;
4. exact vector retrieval over raw episodes;
5. fixed BM25–vector fusion over raw episodes;
6. rolling summary memory;
7. episode retrieval plus direct insight extraction;
8. ELL-Core without concept consolidation;
9. full ELL-Core with evidence-restored learning packets.

The strongest of conditions 3–7 is selected on development data before the sealed run and becomes the confirmatory comparator. This prevents a weak hand-picked baseline from making ELL look effective.

Where reproducible implementations and compatible licences are available, Hindsight, A-MEM, and Reflective Memory Management form the first named comparison tier because they are close to ELL's reflection and structured-memory claim (Latimer et al. 2026; Xu et al. 2025; Z. Tan et al. 2025). GAAMA and PlugMem may be added if their key behaviour can be reproduced (Paul, Sharma, and Sareen 2026; Yang et al. 2026). A method will not be included under another system's name when critical behaviour is missing or reimplemented only approximately.

Separate substrate experiments compare Letta/MemGPT, Mem0, Graphiti/Zep, Hindsight, and TencentDB Agent Memory as context or memory providers. Retrieval-only experiments compare deterministic scan, BM25, exact vectors, HNSW, and TurboVec. Learned operation, experience-graph, skill-evolution, and neural or parametric memory systems are deferred exploratory tracks; they do not enter the first confirmatory comparison.

7.5 Matching and accounting protocol

Every eligible condition receives the same chronological source stream, answer model, answer-generation settings, task opportunities, and maximum answer budget. Retrieval context is capped by the same token budget. Systems may use their native write and retrieval prompts, but every prompt, model call, intermediate artefact, retry, cache policy, and background job is recorded.

Two cost views are reported. Read-path cost measures the user-visible decision. Amortised end-to-end cost includes ingestion, embeddings, extraction, graph or index maintenance, reflection, validation, consolidation, retrieval, and generation over the complete stream. A system is not credited with efficiency by moving work into an unreported background process.

The answer model is prevented from seeing system labels. Where possible, retrieved packets are normalised into a common envelope containing content, timestamps, provenance references, and system-supplied scores. The sealed test is run once per frozen configuration. Failed runs, exclusions, and provider errors remain in the released trace.

7.6 Model conditions

The main reproducibility track uses at least two openly licensed instruction-tuned models from different model families and two capacity bands. Model names are recorded only when the experiment is frozen, because available open models change rapidly. All generation settings, quantisation, serving software, prompts, and hardware are logged.

A hosted-model comparison may be reported separately, but the paper's core claims must remain reproducible without proprietary APIs.

7.7 Metrics

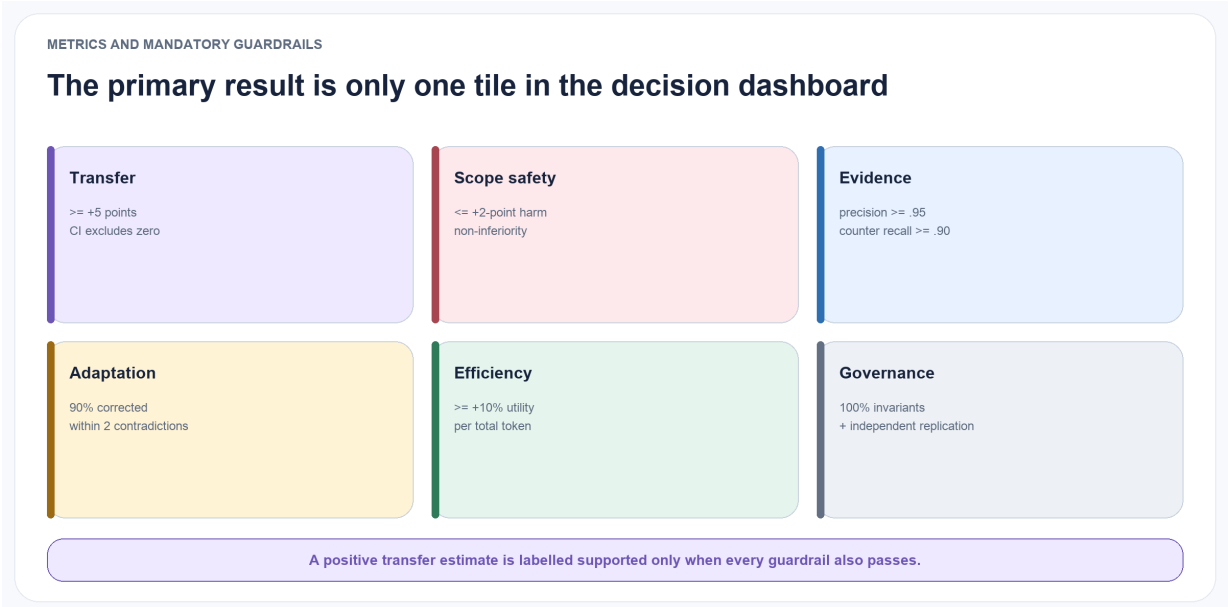

Figure 7.4: **A transfer gain is only one tile in the decision dashboard.** Scope safety, evidence, adaptation, efficiency, governance, and replication remain mandatory rather than optional diagnostics.

Downstream metrics

- task success or benchmark accuracy;
- answer faithfulness to retrieved evidence;
- transfer gain over the episode-only baseline;
- abstention quality;
- action efficiency, including steps to completion.

Concept metrics Concept correctness. Does the induced proposition match the gold latent pattern or human judgement?

Evidence precision. What proportion of cited support actually supports the concept?

Evidence recall. What proportion of relevant support was linked?

Counterevidence recall. Did the system find important exceptions and contradictions?

Scope accuracy. Does the concept apply to the right contexts without over-generalising?

Revision latency. How many contradictory episodes or how much time elapses before a stale concept is contested or revised?

Lineage correctness. Do revisions, merges, and splits preserve accurate parent and evidence links?

Calibration. Does stated confidence correspond to observed concept correctness and application success?

Brier score and expected calibration error will be reported.

Efficiency metrics

- input and output tokens per episode and per decision;
- model calls per accepted concept;
- storage growth;
- median and tail latency;
- energy or hardware-time proxy where measurable;
- task utility per 1,000 retrieved tokens.

Product-facing secondary criteria

- shorthand resolution accuracy without requiring the user to restate stable context;
- precision and recall of proactive just-in-time context, including a penalty for intrusive or irrelevant retrieval;
- useful generalisation to structurally related but lexically dissimilar situations;
- change-detection delay and correction latency after explicit or behavioural counterevidence;
- calibration and selective abstention when evidence is weak, stale, or contradictory;
- explanation faithfulness, citation validity, and user ability to correct, scope, or delete the underlying memory;
- end-to-end task utility per token, latency, storage, model call, and energy or hardware-time proxy, including background consolidation cost;
- zero cross-workspace leakage and complete tested invalidation of deleted evidence from derived projections.

These product-facing measures cannot establish the primary claim. The preregistered decision gates in Section 11 determine whether ELL-Core is supported, partially supported, or rejected before a production memory substrate or learned policy is chosen.

An implementation is not successful merely because it stores more, produces persuasive summaries, or retrieves plausible context.

7.8 Human evaluation

Concept quality cannot be reduced entirely to lexical matching. A stratified sample will be rated by at least three annotators who are blind to the system condition. The rubric covers correctness, support, scope, usefulness, and whether counterevidence was handled appropriately. Inter-rater agreement is reported using Krippendorff's alpha. Disagreements are retained rather than resolved solely by an LLM judge.

LLM-based judges may scale evaluation, but they will be calibrated against the human sample. Prompts and raw judgements will be released. The same model used to generate a concept will not be the only judge of that concept.

7.9 Ablations

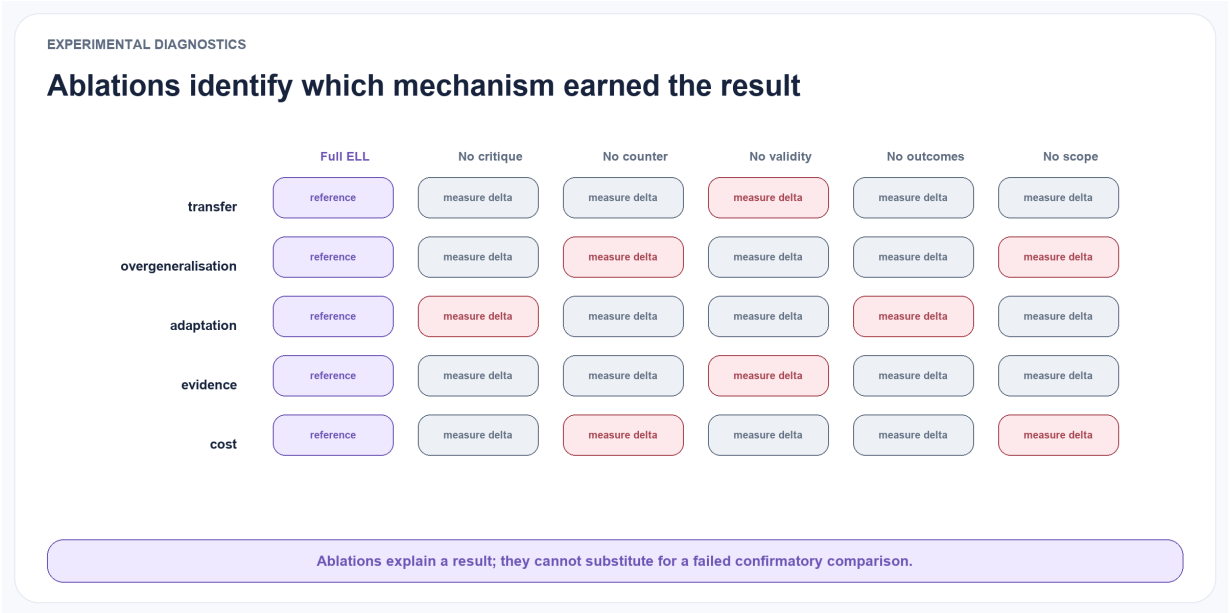

Figure 7.5: **Ablations diagnose which mechanism earned the result.** Removing critique, counterevidence, temporal validity, outcomes, or scope shows how each component affects transfer, overgeneralisation, adaptation, evidence quality, and cost.

The following components are removed one at a time:

- reflection–concept separation;
- counterevidence retrieval;
- evidence diversity threshold;
- temporal validity;
- lifecycle versioning;
- application-outcome feedback;

- source-episode restoration at retrieval;
- graph associations beyond vector similarity.

Additional sweeps vary reflection frequency, promotion thresholds, retrieval budget, and confidence formulation. Architecture ablations compare lexical, vector, graph, temporal, and hybrid association; fixed versus learned extraction and consolidation policy; concept-only versus episode-restored packets; and eager versus event-triggered background processing. This keeps dynamic graph organisation, probabilistic hypotheses, and other research directions behind evidence gates instead of presuming a settled winner.

Safety ablations separately disable deterministic validation around learned write, update, and discard proposals; independent outcome validation around skill evolution; and canonical evidence restoration around neural or parametric adaptation. These conditions are diagnostic and are not deployment configurations.

7.10 Statistical analysis

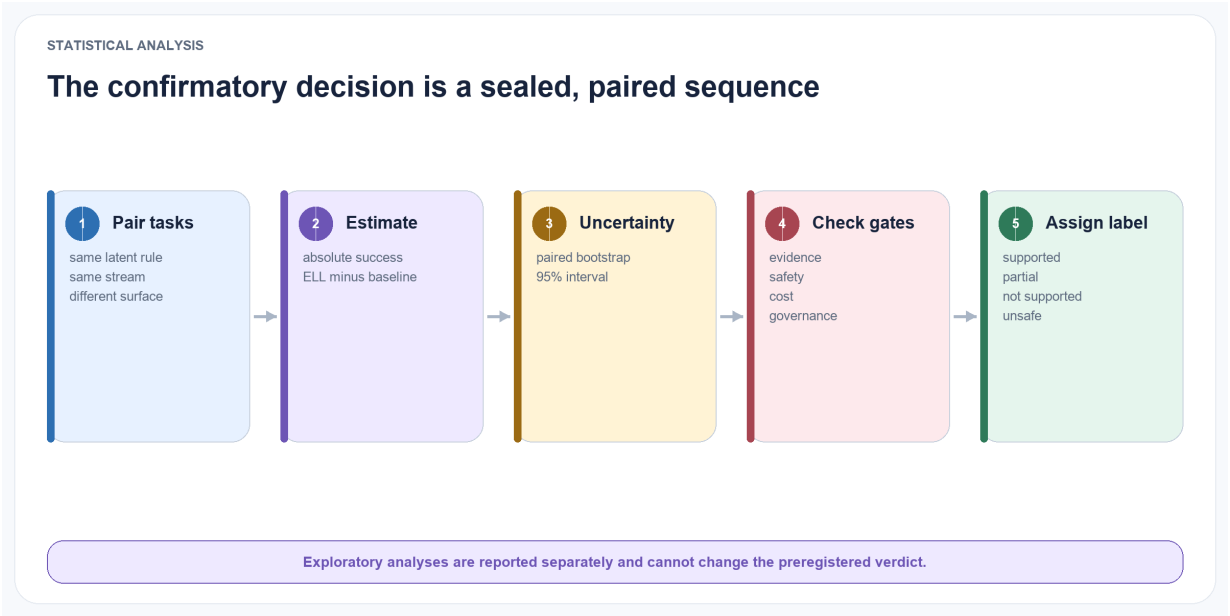

Figure 7.6: **The confirmatory analysis is a sealed, paired sequence.** Matched tasks produce an absolute effect estimate and uncertainty interval, followed by mandatory gate checks and a preregistered outcome label.

The primary binary task outcome uses a paired absolute difference with a paired bootstrap 95% confidence interval; McNemar's test is reported as a sensitivity analysis where appropriate. Continuous or ordinal metrics use paired bootstrap confidence intervals and report effect sizes. Multiple random seeds are used for synthetic generation and stochastic inference. The benchmark size is chosen before the sealed run to provide at least 80% power for the five-percentage-point minimum practically important transfer effect defined in Section 11.

Results are reported by task category and stream length, not only as a single average. Failure analysis separates write, association, reflection, consolidation, retrieval, reasoning, and application errors.

8 Open-Source Research Publication

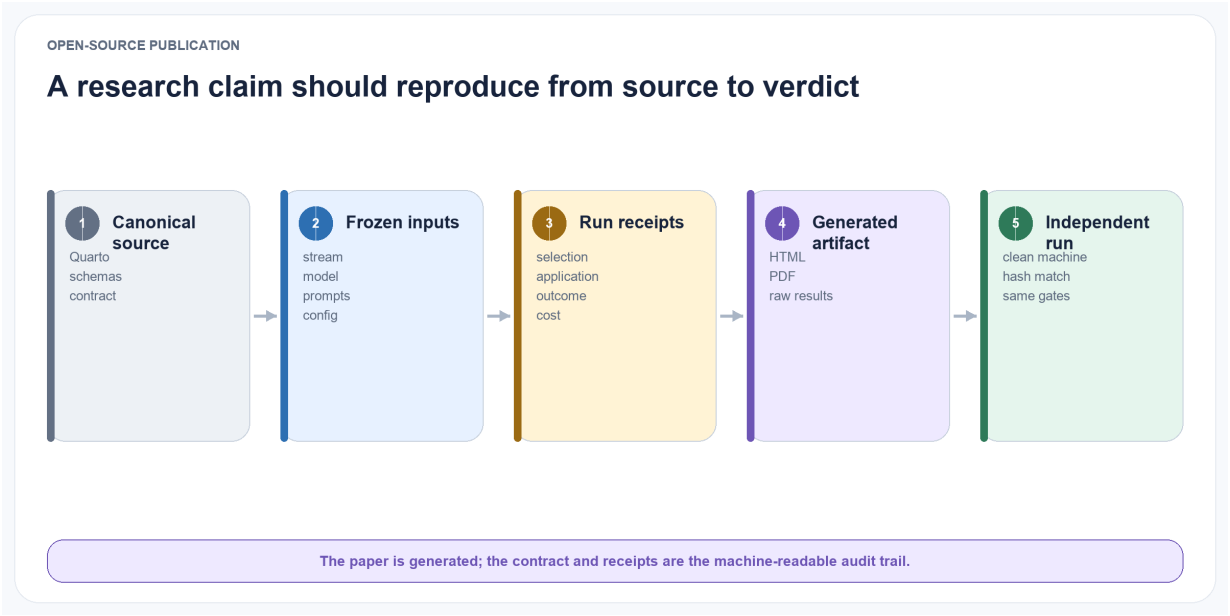

Figure 8.1: **A research claim should reproduce from source to verdict.** Canonical sources, frozen inputs, run receipts, generated editions, and independent runs form one inspectable publication chain.

8.1 Licensing

The repository is released under the permissive MIT License. It covers the paper source, diagrams, documentation, publication tools, and synthetic examples unless a file states otherwise. Third-party datasets retain their original licences and are downloaded separately. A future archival research release should additionally include machine-readable citation metadata and generated dependency attribution.

8.2 Repository structure

```
experience-learning-layer/
  README.md
  LICENSE
  _quarto.yml
  styles.css
  index.qmd
  chapters/
     assets/diagrams/
```

```
examples/
research/
schemas/v0.6/
src/ell/
tests/
pyproject.toml
Makefile
.github/workflows/publish.yml
```

The repository contains the living chapter sources, shared visual diagrams, synthetic lifecycle examples, machine-readable research contract, generated JSON Schemas, deterministic benchmark, in-memory governed core, and the small Quarto configuration required to produce the website and PDF. Generated publication and benchmark output is ignored and rebuilt locally or by GitHub Actions. The reference implementation is explicitly labelled pre-confirmatory so passing software checks cannot be mistaken for evidence that the research hypotheses are supported.

8.3 Reproducibility requirements

Every reported experiment must include:

- immutable configuration files;
- exact Git commit;
- model identifiers and hashes where available;
- prompts and structured-output schemas;
- random seeds;
- hardware and serving configuration;
- raw outputs and failure logs;
- evaluation code and judge prompts;
- token, latency, and storage accounting.

A one-command small-scale reproduction should run on consumer hardware with a compact open model. Full benchmark results may require larger hardware, but the same code path and data format must be used.

8.4 Contribution model

The project begins with a lightweight maintainer model and records its evolution through the manuscript and Git history. Contributions require reproducible publication checks, provenance for example or benchmark changes, and a declaration when generated data or code was produced with model assistance.

Research claims are reviewed separately from software changes. A faster implementation is not accepted as scientifically equivalent unless its behaviour and evaluation remain comparable.

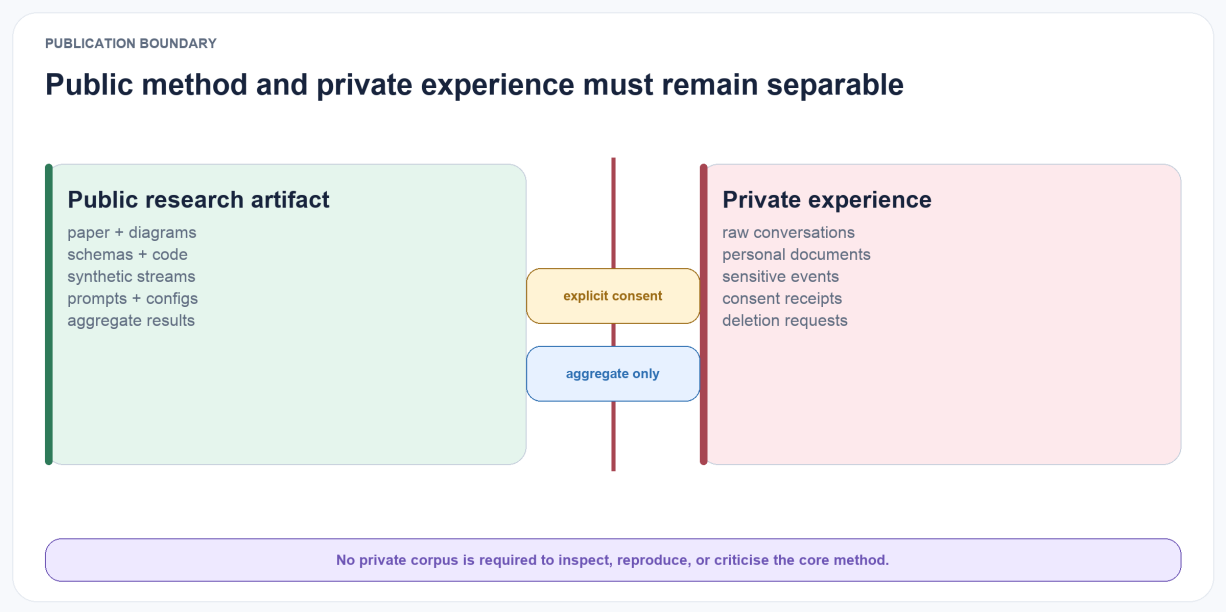

Figure 8.2: **Public method, private experience.** The paper, schemas, code, synthetic streams, prompts, configurations, and aggregate results remain inspectable without publishing raw conversations, personal documents, sensitive events, or consent and deletion records.

8.5 Private data boundary

Private conversation exports can be used locally to test ingestion and qualitative usefulness, but they are not part of the public benchmark and must not be committed. Public experiments use synthetic or properly licensed data. This paper includes data-governance guidance; redaction hooks, local namespaces, and tested deletion cascades are required before any longitudinal pilot.

9 Ethics, Privacy, and Security

A memory system can improve continuity while also increasing risk. Persistent concepts may encode sensitive information, amplify a mistaken interpretation, or influence actions long after the originating interaction.

Long-term memory is therefore a control channel, not merely a convenience layer. Research on trustworthy memory search similarly shows that semantically related memory can still be inappropriate or unsafe for the current task (Zhang et al. 2026).

ELL adopts the following principles.

Visibility. Users should be able to inspect active concepts, evidence, uncertainty, and change history.

Correction. A user correction is stored as high-priority evidence and can trigger immediate contestation, but it should not silently erase historical state when historical reasoning is required.

Deletion. Source deletion propagates to derived concepts. Derived content must not survive simply because it was transformed.

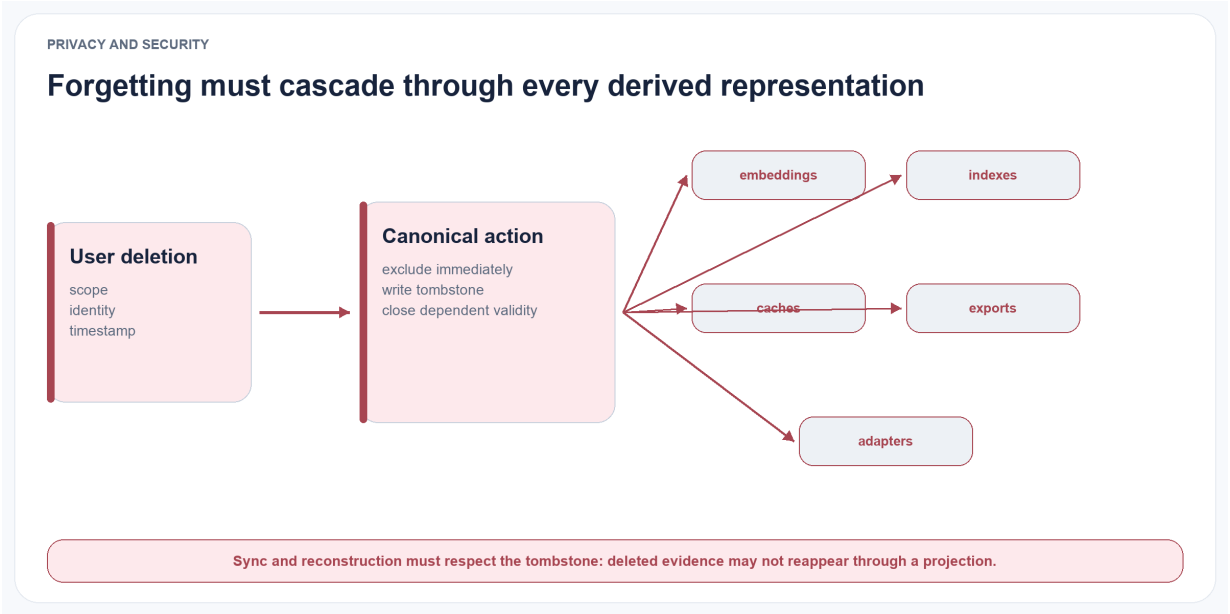

Figure 9.1: **Forgetting is a cascade.** A deletion request immediately excludes canonical evidence, writes a tombstone, closes dependent validity, removes projections, and prevents reconstruction through indexes, caches, exports, adapters, or sync.

Least access. Retrieval enforces source-level permissions and subject namespaces before semantic relevance is considered.

No hidden certainty. Contested or weak concepts are labelled. Confidence and evidence counts are not replaced by persuasive prose.

Poisoning resistance. New memories do not automatically become trusted rules. Promotion requires independent evidence and validators, and sensitive actions can require human approval.

Purpose limitation. Concepts formed for one domain are not automatically reused in another. Cross-domain transfer is an explicit, auditable operation.

Constitutional separation. A learned scaffold may propose bounded changes to reflection, consolidation, retrieval, budgets, retries, or abstention. It cannot change consent, permissions, workspace isolation, sensitivity rules, deletion, provider egress, evaluator isolation, canonical commit authority, or the evidence required to justify a memory. Governance failures disqualify a scaffold regardless of downstream reward.

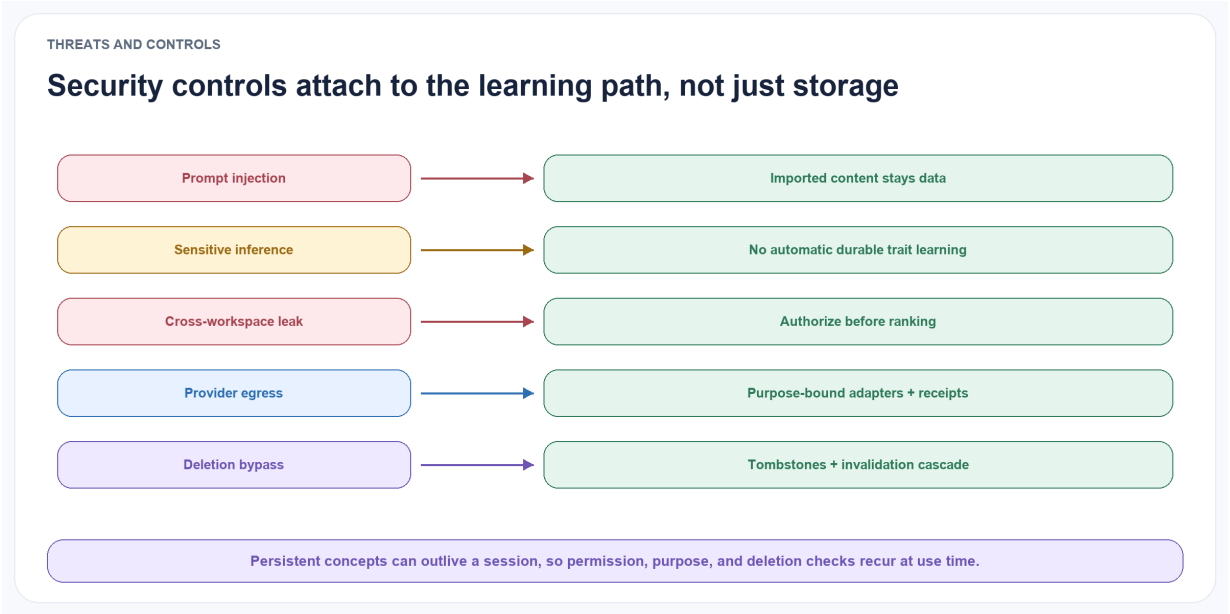

Figure 9.2: **Security controls recur along the learning path.** Imported prompt injection, sensitive inference, cross-workspace leakage, provider egress, and deletion bypass each require a specific control before persistent concepts may influence later decisions.

The system may still infer sensitive traits that were never stated directly. The research implementation therefore defaults to local processing, excludes sensitive-trait inference from public examples, and treats inferred personal concepts as user-governed data. Future work must test whether users can understand and meaningfully control these abstractions.

10 Limitations and Threats to Validity

10.1 Critical appraisal of the approach

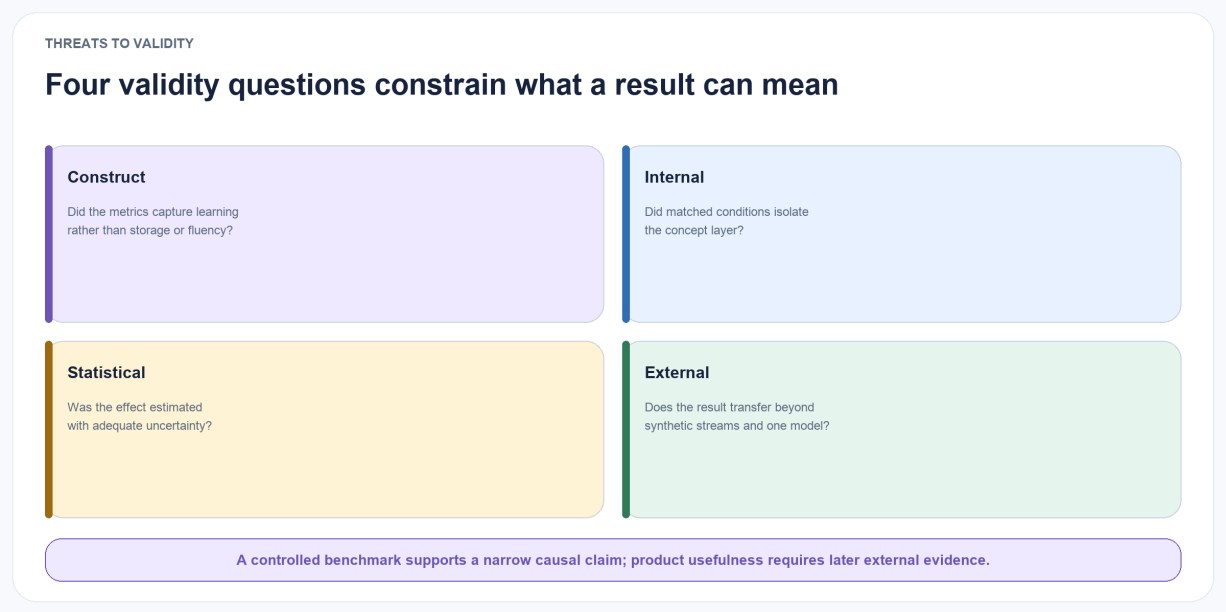

Figure 10.1: **Four validity questions constrain interpretation.** Construct validity asks whether the benchmark measures learning; internal validity asks whether the concept layer caused the effect; statistical validity asks whether the estimate is stable; external validity asks where it generalises.

The strongest criticism of ELL is that it may be an over-engineered combination of ideas that already exist in agent memory, truth maintenance, belief revision, temporal databases, and provenance systems. Reflection, experience abstraction, confidence-bearing opinions, temporal graphs, versioned skills, and evidence tracing all have close precedents. A paper that presents their combination as self-evidently better would be a systems manifesto rather than a research contribution.

The second criticism is attribution. If the first implementation simultaneously adds a graph, vector search, reflection, a concept registry, learned policies, provider adapters, and outcome feedback, an improvement cannot be assigned to the concept lifecycle. The same complexity also creates many tuning degrees of freedom and a large risk of test-set adaptation.

The third criticism is representational favouritism. A synthetic generator with explicit latent concepts naturally rewards a system that stores explicit concepts. Strong raw-retrieval or agent-controlled memory may perform better in open environments where the useful regularity is procedural, perceptual, or difficult to state. External benchmarks and negative-transfer cases are therefore required.

The fourth criticism is causal. An application receipt shows that a concept was present when an action was taken; it does not prove that the concept caused the outcome. The answer model

may ignore the packet, rely on prior knowledge, or succeed for another reason. Factorial packet ablations and randomised inclusion on eligible tasks are needed before making causal utility claims.

These criticisms determine the implementation order: benchmark first, simple baselines second, deterministic ELL-Core third, model-assisted consolidation fourth, and optional infrastructure only after the primary result. The first study has one confirmatory outcome and explicit stop conditions. This discipline is part of the proposal, not an implementation detail.

10.2 Specific threats

First, textual concepts favour knowledge that can be expressed as propositions or instructions. Tacit motor skills, perceptual expertise, and distributed representations may not fit this format. ELL may therefore complement rather than replace parametric or executable skill learning.

Second, provenance does not guarantee truth. Episodes can be incorrect, duplicated, biased, or malicious. A perfectly traceable concept may still be grounded in poor evidence. Source reliability and adversarial robustness require separate study.

Third, LLM-generated reflections may introduce causal stories that are plausible but unsupported. The validation and counterevidence stages reduce this risk but cannot eliminate it. Human evaluation remains necessary for high-stakes domains.

Fourth, the proposed confidence formula is heuristic. Evidence counts are not independent observations, and application outcomes may be delayed or confounded. Calibration must be empirical and domain-specific.

Fifth, synthetic benchmarks can overstate progress if their latent rules resemble the system's representation.

The controlled benchmark is useful for diagnosis but must be paired with natural conversations and agentic tasks.

Sixth, comparisons across memory systems are difficult because ingestion granularity, retrieval budget, model backbone, prompts, and judge choice can dominate results. The protocol therefore fixes budgets and reports complete configurations, but residual implementation differences remain.

Seventh, the architecture adds cost and operational complexity. The system is unsuccessful if the concept layer does not deliver enough transfer, safety, or efficiency to justify reflection, validation, storage, and governance overhead.

Eighth, the term intuition can encourage anthropomorphism or hide uncertainty behind fluent behaviour. In this paper it is only an operational label for measured context selection and adaptation. A compact packet may also compress away rare but important exceptions, so source restoration, counterevidence retrieval, and abstention must be evaluated explicitly.

Ninth, reduced prompt tokens do not necessarily reduce total energy or latency. Background embedding, association, graph maintenance, reflection, and consolidation can move cost out of the visible request path.

Efficiency claims therefore require end-to-end accounting over both write-time and read-time work.

Tenth, learned memory operations introduce reward misspecification and policy-opacity risks. A policy may improve benchmark reward by discarding inconvenient counterevidence, overfitting

retrieval to the judge, or retaining sensitive information. Immutable sources and deterministic commit constraints reduce but do not eliminate this risk.

Self-scaffolding adds a second optimisation surface: a model may learn to change the harness so that tasks become easier to score without becoming more useful. It may expose evaluator artefacts, narrow the attempted task distribution, inflate context or tool use, suppress abstention, or route around evidence checks. ELL therefore fixes the outer environment, tool permissions, governance configuration, sealed data, and evaluator boundary. Deterministic violations make a trajectory ineligible, while a separately frozen language-model judge may veto suspicious intent but never supplies the primary reward. These controls are adapted from Ornith-1.0's self-scaffolding trust boundary, but their effectiveness for longitudinal personal learning remains unproven (Ornith Team 2026).

Personal and organisational outcome streams also create a difficult credit-assignment problem. Rewards may be delayed, sparse, socially biased, or confounded by changes in the user, task, model, or environment. A globally improving scaffold may harm a minority task category, while rapid adaptation may chase noise. Time-forward evaluation, task-category reporting, minimum evidence requirements, staleness-aware policy credit, shadow mode, canaries, and rollback reduce these risks but cannot turn observational feedback into causal proof.

Eleventh, neural and parametric memory may improve adaptation while weakening inspectability and selective deletion. An external provenance ledger can record how an update was produced, but it cannot by itself prove which parameter encodes a fact or that a deletion has removed all influence. Such methods cannot satisfy ELL's canonical governance requirements without new verification techniques.

Finally, cognitive analogies are limited. Human episodic and semantic memory motivate the separation of roles, but ELL should not be interpreted as an account of biological memory or consciousness.

11 Definition of Success and Falsifiability

11.1 Decision rule

ELL-Core succeeds only if it improves future behaviour and passes every mandatory guardrail. Better-looking concepts, persuasive examples, high retrieval recall, or lower foreground prompt cost are insufficient on their own. The following thresholds are minimum practically important effects for the first confirmatory study, fixed before the sealed test is opened.

These thresholds are project decision rules, not universal standards for agent memory. The preregistration must justify them against benchmark variance, task difficulty, and the cost of each error before data collection begins.

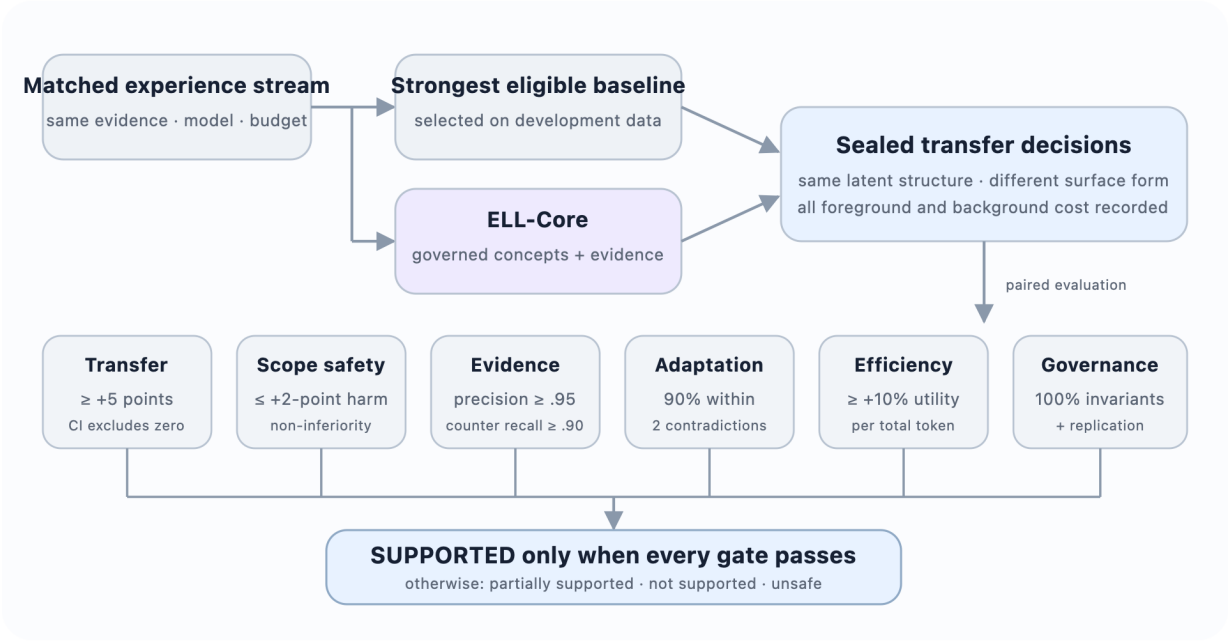

Figure 11.1: **The ELL decision rule.** The same stream and model conditions feed the strongest eligible baseline and ELL-Core. A positive transfer result is reported as supported only when every guardrail also passes.

Gate	Measure	Pass rule
Primary transfer	Paired task-success difference on structurally related, lexically distant sealed cases	Point estimate at least **+5 percentage points** over the strongest eligible baseline and the paired 95% confidence interval excludes zero

Gate	Measure	Pass rule
Unsupported generalisation	Decisions that apply a concept outside its gold or adjudicated scope	Upper bound of the paired 95% confidence interval is less than a **+2-point non-inferiority margin** relative to the comparator
Evidence quality	Precision of cited support and recall of material counterevidence	Support precision at least **0.95** and counterevidence recall at least **0.90** on the controlled benchmark
Change adaptation	Delay after a gold change point or material correction	At least **90%** of affected concepts are contested or revised within two relevant contradictory episodes; post-change stale-guidance rate remains below **5%**
Cost efficiency	Task utility divided by all amortised input and output tokens	At least **10% higher utility per 1,000 total tokens** than the confirmatory comparator, including background work
Governance invariants	Workspace isolation, idempotency, lineage, deletion and projection invalidation	**100% pass** on deterministic and adversarial contract tests; no observed cross-workspace retrieval and no deleted evidence remaining reachable in tested projections
Replication	Sensitivity to model and task family	Primary transfer and unsupported-generalisation gates pass with two open model families; the transfer direction is also positive on at least one external cognitive or action benchmark

The frozen v0.6 contract uses 640 paired sealed tasks per confirmatory condition. This exceeds the approximate 625 pairs required for 80% power under a two-sided 0.05 test, a five-point effect, and an assumed 0.20 discordant-pair rate. The primary interval is a 10,000-resample paired percentile bootstrap with a committed analysis seed. Predeclared corrupt source artifacts and duplicate task identifiers are the only eligible exclusions; provider failures remain scored failures with their traces retained.

11.2 Outcome labels

Supported. Every gate passes, including the external replication gate. The paper may conclude that ELL-Core provides evidence for a useful governed concept layer under the tested conditions. A positive Phase 4 result before external replication is labelled *provisionally supported*, not supported.

Figure 11.2: **The verdict reports both benefit and safety.** Supported requires every gate; partially supported names the failed gate; not supported redirects work to the stronger or simpler method; unsafe disqualifies deployment.

Partially supported. Transfer improves, but one or more safety, adaptation, efficiency, governance, or replication gates fail. The paper must name the failed gate and cannot recommend the full architecture for deployment.

Not supported. The primary transfer gate fails, or the strongest simple baseline matches ELL at lower cost. Work should then focus on the winning simpler method, such as evidence-grounded retrieval, agent-controlled search, or direct insights.

Unsafe. A governance invariant fails. The affected configuration is ineligible for deployment regardless of average task performance.

11.3 Product success is a later claim

After the confirmatory result, a product study may test whether people experience less repetition, better shorthand understanding, timely context, rapid correction, and trustworthy control. Product success requires blinded task evaluation and user research; it cannot be inferred from synthetic benchmark accuracy. The word *intuition* remains a product-level description of these behaviours, not the primary scientific outcome.

11.4 Explicit rejection conditions

The central approach is weakened or rejected if:

- direct insight extraction or agent-controlled flat memory matches ELL at lower cost;
- concept metrics improve without improving held-out decisions;
- concept packets omit exceptions that raw episode retrieval preserves;
- gains disappear under equal total-compute accounting;
- revision causes unstable concept churn or slow recovery after change;

- results depend on one model family, one generator, or one evaluator;
- the system cannot invalidate derived state after deletion; or
- outcome feedback strengthens concepts without independent evidence that their use helped.

Negative results remain valuable. They can show that explicit concepts are unnecessary for some task classes, that consolidation should be triggered only by failures, or that governed evidence retrieval is more useful than concept formation.

12 ELL Research and Implementation Plan

12.1 Planning rule

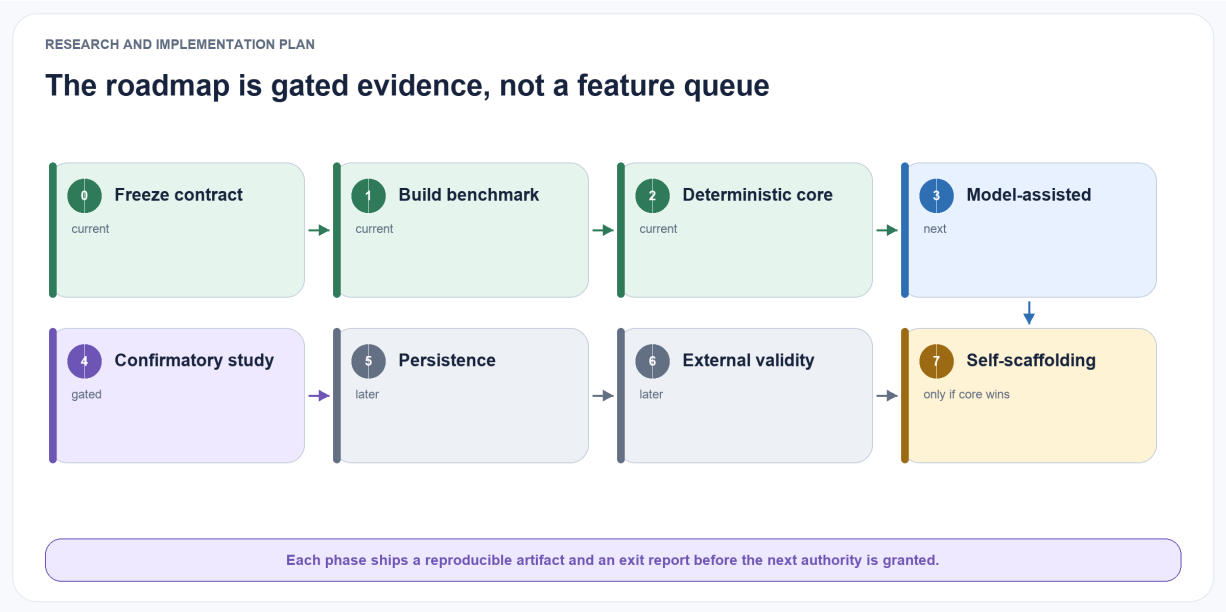

Figure 12.1: **The roadmap is gated evidence, not a feature queue.** Phases 0-2 establish the contract, benchmark, and deterministic core; model-assisted learning comes next; the confirmatory study gates persistence, external validity, and any self-scaffolding authority.

ELL is built in evidence-producing vertical slices. Each phase must leave a reproducible artefact, an exit report, and a go, revise, or stop decision. The paper repository remains publication-first; executable packages may be versioned separately and linked by immutable release identifiers so implementation churn does not obscure the manuscript.

12.2 Phase 0 — Freeze the research contract

Current status: implementation-complete release candidate. Machine-readable contracts, schemas, worked adversarial examples, and local verification exist; an immutable v0.6 tag or archival release is still required for formal exit.

Deliverables:

- publish paper v0.6 with the narrowed ELL-Core boundary and separately gated self-scaffolding track;
- freeze the primary estimand, success gates, baseline eligibility rules, and cost-accounting schema;

- publish the source, episode, reflection, concept, evidence-link, application, outcome, and audit schemas as specifications;
- publish worked examples for support, counterevidence, revision, deletion, and unsafe retrieval.

Exit criterion: an independent reader can identify exactly what result would support or reject ELL and can rebuild the HTML and PDF from the same sources.

12.3 Phase 1 — Build the benchmark before ELL

Current status: executable deterministic reference slice. The generator, sealed-development boundary, seven baselines, receipts, manifests, and local reproducibility tests exist. Two independent clean-machine reproductions and the frozen stochastic conditions remain required for formal exit.

Deliverables:

- implement deterministic latent-pattern streams with paraphrase, distractors, exceptions, correlated evidence, change points, delayed outcomes, permissions, and deletion events;
- create chronological train, development, and sealed test partitions;
- implement immutable run manifests and application receipts;
- implement no-memory, maximum-context, BM25, exact-vector, fused retrieval, rolling-summary, and direct-insight baselines;
- run a development-only power analysis and freeze the sealed configuration.

Exit criterion: two clean-machine runs reproduce the same dataset hashes and deterministic baseline metrics, and stochastic conditions reproduce within declared intervals. If the benchmark cannot distinguish known-good from deliberately broken policies, stop and repair the benchmark.

12.4 Phase 2 — Implement deterministic ELL-Core

Current status: executable in-memory reference slice. Lifecycle, provenance, valid and observed time, deletion cascades, idempotency, purpose grants, and workspace isolation have deterministic tests. Broader property/adversarial runs and a formal invariant exit report remain required.

Deliverables:

- implement typed boundary schemas, an in-memory append-only store, deterministic identifiers, and an evidence ledger;
- implement concept lifecycle transitions, valid-time and observed-time handling, idempotency, workspace isolation, and deletion invalidation without an LLM;
- use fixture reflections and concepts to test the entire application–outcome loop;
- add property and adversarial tests for lineage, retries, corrections, tombstones, and cross-workspace access.

Exit criterion: all governance invariants pass and every derived object resolves to permitted source spans. No graph database, hosted memory provider, approximate vector index, or learned policy is permitted in this phase.

12.5 Phase 3 — Add model-assisted reflection and consolidation

Deliverables:

- add structured-generation adapters for at least two open model families;
- quarantine generated candidates before deterministic validation and commit;
- implement reflection, counterevidence search, concept proposal, merge, split, revision, and evidence restoration;
- run same-model, cross-model, deterministic-validator, and human-review conditions;
- execute the preregistered component ablations on development data, then freeze prompts and policies.

Exit criterion: the sealed ELL-Core comparison can run without manual intervention and emits complete evidence, decision, outcome, and cost traces. A failed governance invariant blocks the sealed run.

12.6 Phase 4 — Run the confirmatory study

Current status: blocked, with the frozen runner implemented. Comparator selection, paired intervals, gate calculations, and verdict labels are executable, but the runner refuses evaluation until Phase 0–3 evidence, frozen prompts and policies, and a single authorized sealed run exist. No confirmatory verdict has been issued.

Deliverables:

- select the strongest eligible baseline using development data only;
- execute the sealed comparison once per frozen model and seed condition;
- publish all gate results, confidence intervals, exclusions, failures, and raw receipts;
- issue a supported, partially supported, not supported, or unsafe verdict using Section 11 without post-hoc threshold changes.

Exit criterion: the result can be reproduced from a tagged benchmark, implementation, configuration, and paper release. Phase 4 may issue a provisional verdict; the final *supported* label remains reserved until the external replication gate in Phase 6 passes. If the primary gate fails, stop concept-layer expansion and investigate the winning simpler baseline.

12.7 Phase 5 — Evaluate persistence and replaceable substrates

This phase begins only if Phase 4 supports continued work.

Current status: pre-confirmatory conformance tooling only. In-memory and SQLite canonical semantics plus lexical and exact-vector projection rebuild/deletion checks are implemented. TurboVec and external-memory adapters remain unavailable and cannot write canonical state. The phase is not formally eligible without a qualifying Phase 4 result.

Deliverables:

- add SQLite plus full-text search as the first durable local adapter;
- compare exact vectors, HNSW, and TurboVec under identical candidate and permission filters;
- reproduce selected Hindsight, TencentDB Agent Memory, Mem0, Zep/Graphiti, and Letta conditions through narrow adapters where their public interfaces and licences permit;

- verify rebuildability, crash recovery, deletion propagation, provider egress controls, and cost accounting.

Exit criterion: replacing a substrate does not change canonical identity, lifecycle, permissions, or evaluation semantics, and any performance claim is tied to a reproducible adapter version.

12.8 Phase 6 — External validity and product pilot

Current status: package-validation adapters and a consent authority are implemented, but no external dataset or participant has entered a run. Local packages require exact hashes, citations, versions, and declared licences. A pilot requires protocol readiness plus signed, time-bounded, purpose-specific consent and tested withdrawal.

Deliverables:

- run LoCoMo-Plus and at least one memory-dependent action benchmark such as Mem2ActBench or MemoryArena;
- complete blinded human evaluation of concept support, scope, usefulness, and correction;
- test shorthand, proactive relevance, correction, explanation, and control in a consented small pilot;
- measure total system cost, user-perceived intrusiveness, and failure recovery.

Exit criterion: the transfer direction survives at least one external benchmark, no mandatory safety gate regresses, and users can inspect, correct, scope, and delete learned state. Only then should a product client or broader connector programme be planned.

12.9 Phase 7 — Evaluate governed self-scaffolding

This phase begins only if the ELL-Core result supports continued work and Phase 6 finds no external-validity or user-control failure that would make adaptive policy learning premature.

Current status: research specification only. No `LearningScaffold` schema, proposal controller, scaffold trial, shadow deployment, canary, or promotion authority is implemented, and no self-scaffolding result is claimed.

Deliverables:

- specify versioned `LearningScaffold`, `ScaffoldProposal`, `ScaffoldTrial`, and `ScaffoldDecision` contracts;
- record the exact scaffold, model, prompt, tool, policy, evidence, action, outcome, and cost lineage for every eligible application;
- compare frozen, development-tuned, governed model-mutated, and—if justified by outcome volume—adaptive controller conditions under equal budgets;
- implement immutable outer-boundary checks, deterministic disqualification, a separately frozen judge veto, chronological replay, out-of-time evaluation, shadow mode, category-limited canaries, and rollback;
- evaluate task utility, transfer, adaptation, calibration, negative transfer, policy stability, safety-veto rate, deletion behaviour, promotion error, rollback reliability, and total cost;
- publish all candidate mutations, hard-gate results, exclusions, failures, and scaffold-version decisions.

Exit criterion: a self-scaffolding condition reproducibly improves the preregistered objective over the strongest frozen scaffold on time-forward and external tasks without regressing any mandatory governance or user-control gate. Promotion and rollback must both reproduce from immutable receipts. If no adaptive condition clears that standard, ELL retains frozen task-category scaffolds.

12.10 Deferred research tracks

Governed self-scaffolding is the explicit Phase 7 follow-on rather than part of ELL-Core. Autonomous skill evolution, experience graphs, neural or parametric memory, multimodal capture, sync, and hosted memory services remain separate experiments. Each must justify itself through a preregistered incremental comparison; none inherits support merely because ELL-Core or self-scaffolding succeeds.

13 Deferred Experience Learning Layer Expansion

The research architecture above asks whether evidence-grounded concepts improve transfer and future behaviour. This chapter records the broader product and research horizon, not the Phase 1 implementation plan. None of these extensions enters the confirmatory comparison unless ELL-Core first clears the gates in Section 11.

The expanded architecture generalises the initial scaffold into an open, local-first experience-learning layer usable by people, applications, and AI agents. It does not replace the original empirical question.

L is not primarily a chatbot, note-taking application, Zettelkasten, vector database, or graph visualisation. Its north star is to give any AI a natural, evolving intuition about the user or organisation—grounded in experience, economical to use, sensitive to change, explainable when inspected, and always under user control. Its learning loop captures an experience, preserves its source, extracts typed candidates, decides under deterministic policy what may become durable, consolidates related memories without losing contradictions, retrieves the smallest useful context, observes outcomes, and revises future behaviour. Models interpret meaning and propose. Deterministic services validate, authorize, version, and commit.

Intuition is an emergent capability of this loop, not another row type or a claim that the system resembles a person. Evidence-grounded compression, prediction, association, relevance selection, temporal adaptation, and outcome feedback should together support shorthand, proactive just-in-time context, useful generalisation, rapid correction, change detection, calibrated uncertainty, and optional explanation. The delivery object is a compact intuition packet with sufficient evidence and uncertainty to be useful, plus stable references for deeper inspection.

13.1 System layers and technology boundaries

The expanded architecture has three independently evolvable layers:

- input and capture adapters turn conversations, recordings, documents, application events, and future connectors into immutable source artifacts, normalized events, and bounded episodes;
- memory and retrieval infrastructure provides replaceable persistence, search, association, graph, vector, or hosted-memory projections;
- the canonical learning layer evaluates evidence, maintains temporal and probabilistic hypotheses, governs durable concepts, records applications and outcomes, and controls revision, contradiction, and forgetting.

None of these layers is implemented in the current paper repository. SQLite, hosted memory services, vector indexes, and graphs remain candidate adapters or projections to integrate and compare. The canonical learning layer is specified through proposed lifecycle contracts, but those

contracts are not yet an operational closed learning loop. This distinction prevents an external memory system from silently becoming L's truth or policy authority when implementation begins.

The proposed direction is immutable evidence plus dynamic, temporal, probabilistic concepts and hypotheses. Extraction, association, consolidation, retrieval, and forgetting policies should be pluggable and may become model-driven or learned. Their proposals remain subject to deterministic provenance, permissions, workspace isolation, deletion, schema, and canonical commit governance. No fixed graph, vector method, note-linking scheme, or model family is selected as the final architecture before the success criteria and ablations in Section 7 are run.

13.2 Plural memory semantics

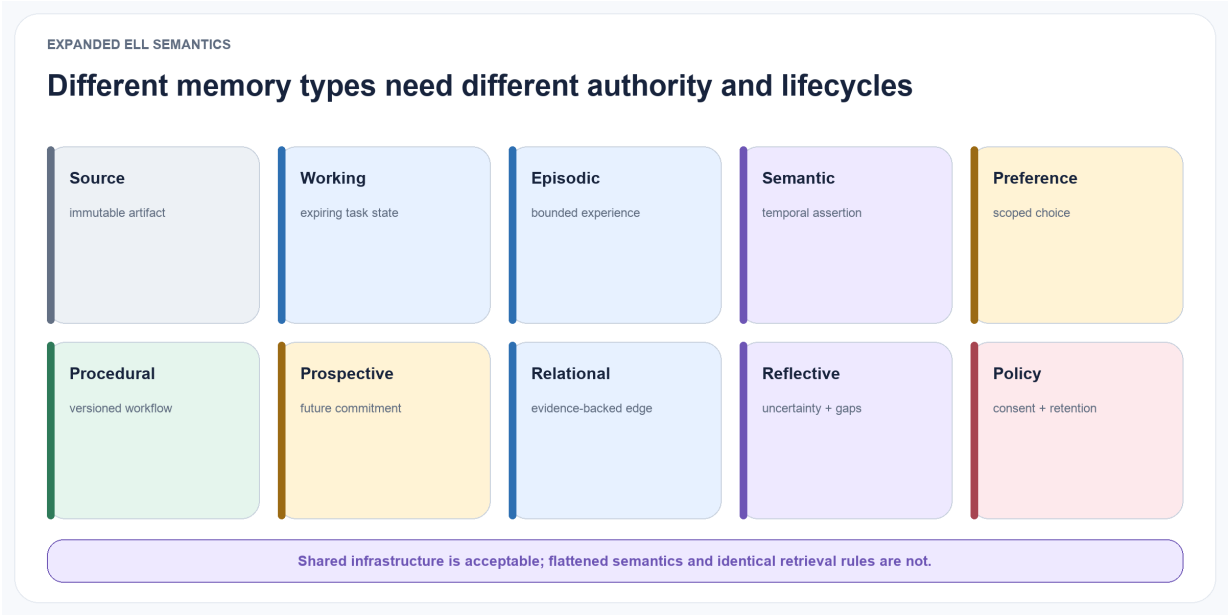

Figure 13.1: **Memory is plural, not one untyped table.** Source, working, episodic, semantic, preference, procedural, prospective, relational, reflective, and policy memory require different authority, lifetime, and retrieval rules.

The v0.1 paper distinguishes episodes, reflections, concepts, applications, and outcomes. The broader ELL architecture retains these objects while distinguishing additional memory semantics that have different authority, lifetime, and retrieval rules:

- source memory preserves immutable artifacts and exact addressable spans;
- working memory holds expiring state for the current task or conversation;
- episodic memory represents bounded experiences, decisions, actions, and outcomes;
- semantic memory represents temporal assertions rather than timeless key-value pairs;
- preference memory represents scoped choices, exceptions, strength, and explicitness;
- procedural memory represents versioned workflows, preconditions, approvals, evidence, and rollback;
- prospective memory represents commitments, reminders, deadlines, and unresolved threads;
- relational memory represents evidence-backed temporal edges;
- reflective memory records uncertainty, freshness, usefulness, contradictions, and knowledge gaps;
- policy memory defines retention, consent, provider, sharing, and deletion boundaries and is never ordinary model context.

These forms may share infrastructure, but they must not be flattened into one untyped table with identical lifecycle semantics. The useful layer is the explainable connection between evidence and future behaviour, not a decorative node cloud.

13.3 Source, event, and episode boundary

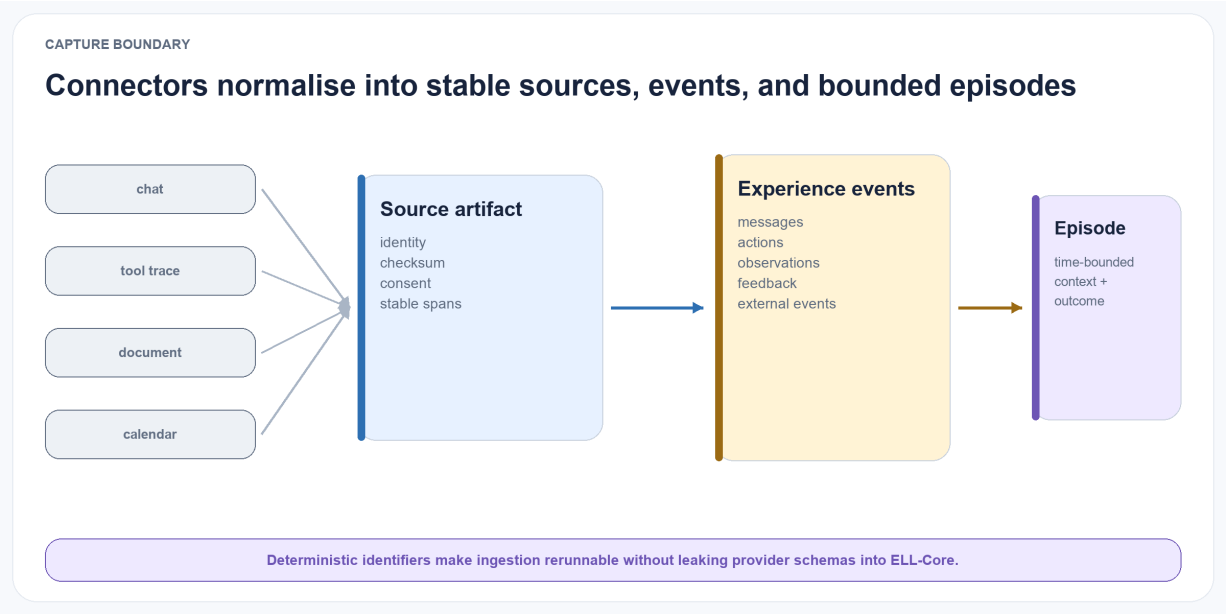

Figure 13.2: **Provider connectors end at a stable normalisation boundary.** Chats, tool traces, documents, and calendars become immutable source artifacts, typed experience events, and time-bounded episodes with deterministic identifiers.

All provider connectors terminate at a common normalization boundary. An immutable SourceArtifact preserves source identity, checksums, timestamps, sensitivity, consent, and stable spans. Normalized ExperienceEvents represent user messages, assistant messages, tool calls and results, file changes, decisions, feedback, and external events. One or more events form a bounded Episode containing inputs, responses, actions, observations, and outcomes.

Stable deterministic identifiers make ingestion rerunnable. The same connector, external reference, source version, and source event produce the same domain IDs. Conversation exports, tool traces, calendar records, or future connectors can therefore be replaced without placing their schemas inside reflection, consolidation, policy, or retrieval services.

13.4 Governed candidate and commit path

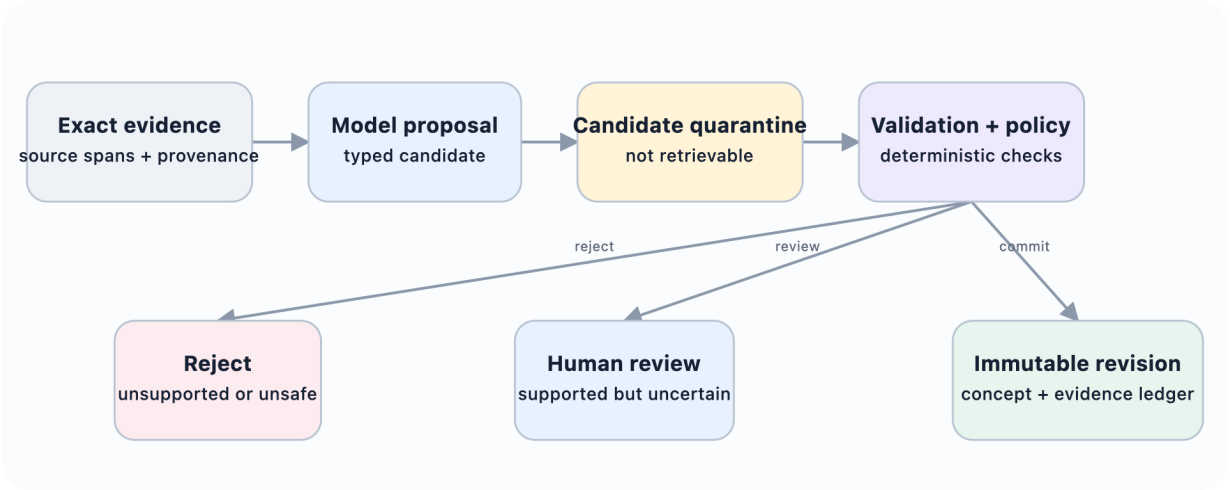

Figure 13.3: **Models interpret; deterministic code governs.** Generated interpretations enter quarantine. Schema validation and deterministic policy decide whether they are rejected, reviewed, or committed.

A model response is untrusted structured input. It enters a CandidateMemory quarantine and cannot participate in normal retrieval. Deterministic validation checks the schema, cited source and span existence, workspace access, sensitivity inheritance, temporal fields, allowed predicates, and referenced memories. Policy then selects one of three initial outcomes:

- explicit user statements, corrections, and user-confirmed candidates may commit after validation;
- supported ordinary inferences and inferred procedures wait for review;
- unsupported non-explicit claims and sensitive model inferences are rejected.

The commit service is the sole canonical writer. It uses idempotency keys and optimistic concurrency, writes an immutable revision, appends a content-minimized audit event, and schedules disposable projections. A correction creates a new memory and closes the validity of the superseded revision. It never rewrites history. A lower-authority inference cannot supersede explicit user evidence.

13.5 Policy-bound evidence retrieval

Retrieval is a typed service rather than direct database or vector-index access. A request includes actor, purpose, workspace, scope, allowed memory types, maximum sensitivity, evidence preference, and a fixed context budget. Authorization and lifecycle filtering happen before relevance ranking. Superseded, forgotten, expired, cross-workspace, and sensitivity-incompatible records do not enter ordinary results.

Candidate generation may combine lexical search, vectors, relation neighbourhoods, time, pinning, procedures, and later learned rerankers. Exact-vector, HNSW, graph, and hosted memory adapters remain replaceable projections to benchmark; no index or provider is canonical memory. The response is an EvidencePacket, presented at the application boundary as an intuition packet,

containing selected current claims, scoped preferences, episodes, procedures, commitments, citations, selection explanations, uncertainty, and known material contradictions. A compact concept never becomes a naked instruction detached from its source.

Concrete comparison adapters may wrap Hindsight's structured networks, Graphiti/Zep temporal graphs, Mem0 extraction and retrieval, Letta-style context management, or TencentDB Agent Memory's layered Chat Memory, Skill, Wiki, and CodeGraph assets. Exact search, HNSW, and TurboVec remain alternative retrieval projections beneath the same policy filter. Learned-policy adapters may later implement AgeMem-style tool actions, AtomMem-style atomic operations, or MemSkill-style evolving routines. Experience and skill adapters may provide EXG-style graphs, ReasoningBank-style strategies, or lifecycle-managed skill candidates. Neural accelerators may provide Titans-, HOPE-, or TMEM-style state. In every case, the adapter returns candidates or acceleration state; canonical source, concept, application, outcome, and deletion records remain governed by ELL.

13.6 Local-first and provider-neutral topology

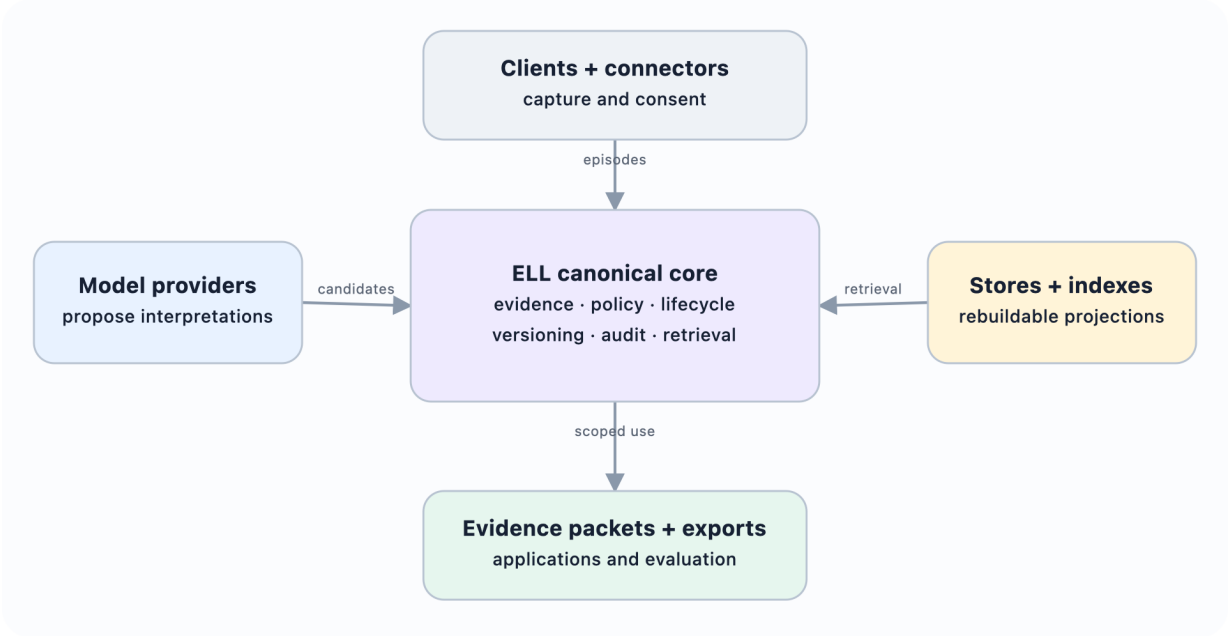

Figure 13.4: **Canonical learning stays provider-neutral.** Clients, model providers, stores, and indexes are replaceable adapters. Canonical identity, evidence, policy, lifecycle, and audit remain inside ELL.

A complete usable state can live on one device. Network absence must not prevent capture, browsing, correction, or lexical retrieval. Remote processing visibly degrades to local processors or queued work. A practical reference adapter may use SQLite, content-addressed encrypted files, FTS5, and a replaceable vector index, but these technologies do not define the domain.

Four authentication concerns remain separate: L user identity, workspace authorization, model-provider credentials, and connector grants. A model adapter may call a provider API; a client or agent may call L through a narrow protocol; or L may export a scoped context bundle. Provider credentials never define L identity, canonical records, or access policy.

Forgetting immediately excludes a record, writes a tombstone, removes projections, and triggers evidence-aware invalidation. Sync must not resurrect tombstoned content. Sensitive trait inference is disabled for automatic durable learning. Imported documents are untrusted content, never system instructions. Secrets do not enter prompts, canonical memory, exports, analytics, or logs.

14 Research Artifact Status

The repository contains the publication plus a deterministic Phase 0–2 reference artifact. It does not yet contain model-assisted learning, governed self-scaffolding, or confirmatory results. The implementation makes the proposed contracts executable without presenting passing unit tests as evidence for H1–H7.

Figure 14.1: **Implemented slices are not empirical support.** The repository currently demonstrates contracts, streams, baselines, and deterministic ELL-Core behaviour. Later-phase support code does not establish a confirmatory verdict, external validity, or self-scaffolding.

14.1 Canonical and generated editions

The Quarto Markdown chapters are the sole canonical publication source. The versioned JSON research contract and schemas are the canonical machine-readable experiment boundary. A single `quarto render` derives the multi-page HTML reading edition and downloadable PDF and copies the contract artifacts into the published site. Quarto provides navigation, search, section numbering, cross-format links, and publication metadata. Generated output is ignored by Git and rebuilt locally or by the GitHub publishing workflow, keeping ordinary paper edits small and reviewable.

14.2 Synthetic lifecycle examples

The repository includes a small synthetic JSONL collection covering explicit scoped preferences, single-event inference, unsupported claims, sensitive inference, explicit correction, material con-

tradiction, temporal change, Dutch evidence, prompt injection embedded in imported content, deletion cascades, permission revocation, derived-state invalidation, and cross-workspace retrieval. These cases make proposed policy outcomes concrete for readers. They are examples, not benchmark results, and no empirical result is inferred from them.

14.3 Executable reference slices

Draft 2020-12 JSON Schemas are generated from strict runtime contracts for the eight canonical objects, cost and run traces, learning packets, invalidation reports, evaluator judgments, and benchmark records. A machine-readable v0.6 research contract freezes the estimand, gates, baseline-selection rule, power assumptions, exclusions, sealed-seed handling, and cost boundary.

The Phase 1 slice generates deterministic 50, 200, and 1,000-record chronological streams with 30, 120, and 640 paired tasks. Streams include support, contradiction, scoped exceptions, correlated evidence, change points, delayed outcomes, permission denials, deletions, distractors, and sealed-seed commitments. It implements no-memory, maximum-context, BM25, exact-vector, fused-retrieval, rolling-summary, and direct-insight baselines. Every task produces a run manifest, selected-record trace, application receipt, evaluator judgment, and cost trace. Development generation does not materialise the sealed partition.

The Phase 2 slice provides an in-memory deterministic ELL-Core. It validates exact source spans, requires quarantined reflections to pass review, commits immutable concept versions with evidence links, preserves revision lineage, enforces workspace and purpose grants before retrieval, records applications and independently sourced outcomes, rejects conflicting idempotent retries, and invalidates dependent canonical and projection state after deletion. It has no LLM, graph, vector service, hosted memory, or learned policy dependency.

14.4 Remaining gates

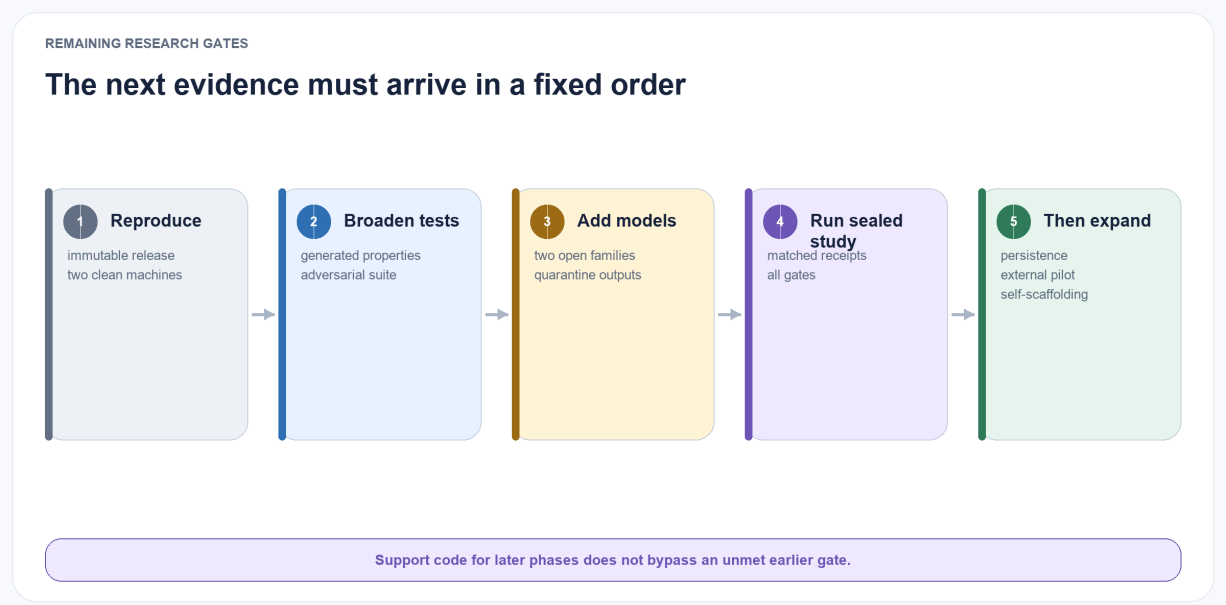

Figure 14.2: **The next evidence must arrive in order.** Reproducible releases and broader adversarial tests precede model-assisted generation; frozen model conditions precede the sealed study; persistence, external pilots, and self-scaffolding remain downstream.

Phase 0 still requires an immutable v0.6 tag or archival release. Phase 1 requires two independent clean-machine reproductions and stochastic model conditions within declared intervals; the current deterministic tests are necessary but not that independent evidence. Phase 2 requires broader generated/property testing and an exit report showing every derived object resolves to permitted source evidence across the full adversarial suite. Persistent-store rebuilds, provider egress enforcement, job recovery, and substrate crash recovery remain Phase 5 concerns rather than prerequisites for the in-memory Phase 2 slice.

Phase 3 is the next implementation step only after those exits are independently reviewed. It adds structured generation from two open model families, keeps every output quarantined until deterministic validation, and freezes prompts and policies on development data before any sealed run.

Phase 4–6 support code is present without bypassing that sequence. The confirmatory runner returns a blocked report until the frozen prerequisites and matched sealed receipts exist. The substrate suite currently compares in-memory and SQLite canonical semantics plus lexical and exact-vector projection behavior; optional TurboVec and external-memory adapters are unavailable and barred from canonical writes. External benchmark adapters require local licence-declared, hash-matched packages, while the pilot authority requires a ready protocol and active purpose-specific consent. No Phase 4 verdict, Phase 5 performance comparison, external benchmark result, or human-pilot claim exists.

Phase 7 is a research specification only. The paper defines the proposed scaffold objects, immutable outer boundary, replay and time-forward comparison, shadow and canary sequence, promotion and rollback rules, and mandatory failure gates. No scaffold schema, proposal controller, trial runner, adaptive policy, or deployment authority has been implemented.

The project does not advance merely because an implementation exists. It advances when the primary transfer result and every mandatory guardrail in Section 11 pass. If a simpler approach matches ELL at lower cost, the research plan explicitly adopts that simpler method and records the concept-layer hypothesis as unsupported for the tested conditions.

15 Conclusion

The Experience Learning Layer begins from a simple distinction: access to old experience is not the same as learning from it. Current research already demonstrates reflection, dynamic memory organisation, schema induction, concept graphs, and semantic or procedural abstraction. The next useful contribution is therefore a stricter account of how an abstraction earns trust, remains connected to evidence, changes when challenged, and improves future behaviour.

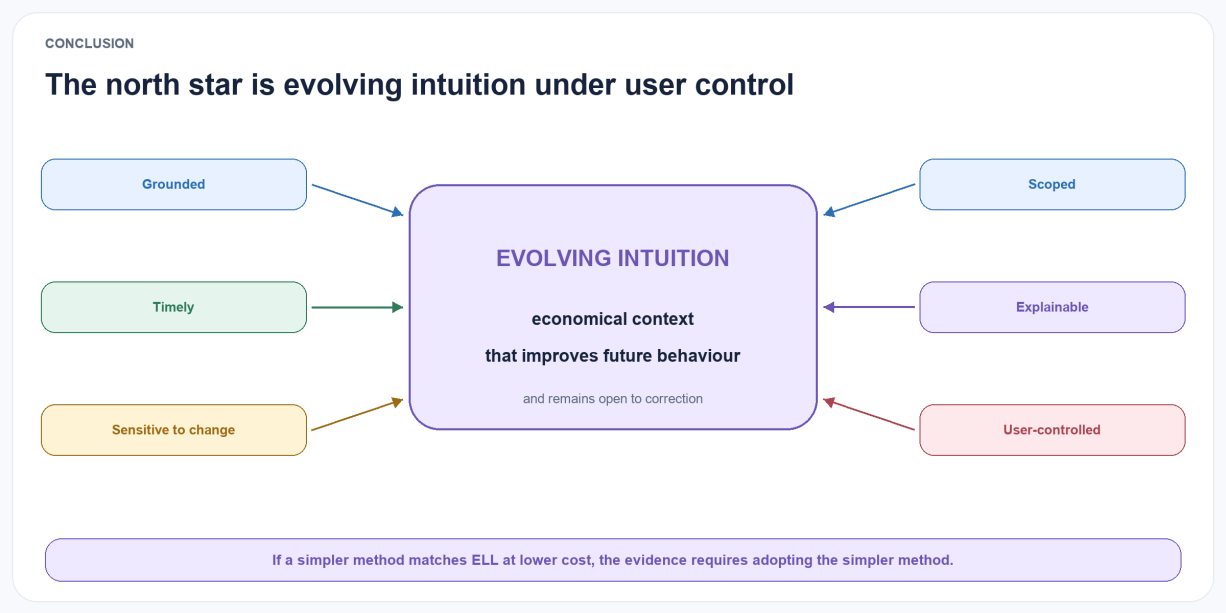

Figure 15.1: **The ELL north star.** Evolving intuition means economical context that is grounded, timely, scoped, sensitive to change, explainable, and user-controlled. It remains a falsifiable product direction, not an anthropomorphic claim.

ELL proposes an explicit path from episodes to provisional reflections, from reflections to versioned concepts, and from concepts to measured applications and outcomes. The architecture preserves both support and counterevidence, treats scope and temporal validity as first-class fields, and makes revision reversible and auditable. Its value will be determined empirically, not by the plausibility of the design.

The practical expression of that hypothesis is economical, timely context that helps an AI understand shorthand and adapt to change without hiding where its guidance came from. The scientific claim is narrower: under matched models, streams, and total compute, ELL-Core must improve sealed structurally distant transfer by at least five percentage points over the strongest eligible baseline while passing every safety, evidence, adaptation, cost, governance, and replication gate. The term *intuition* is deferred to later product research.

If ELL-Core earns that foundation, the next learning question is whether ELL can improve not only its concepts but the scaffold by which it forms and applies them. The proposed second-order loop treats reflection, evidence selection, consolidation, retrieval, budgets, retries, and abstention as versioned experimental policy while keeping provenance, consent, permissions, deletion, canonical commits, evaluators, and rollback outside learned control. This is a separate falsifiable

programme, not permission for unrestricted self-modification and not a way to rescue a failed core result.

The project is open by default. The paper, diagrams, synthetic examples, and reproducible reading editions make the proposal easy to inspect, criticise, and extend. The immediate next step is the benchmark, not a product or a more elaborate architecture. If direct insight extraction, agent-controlled flat memory, or evidence-grounded retrieval matches ELL at lower cost, that result should replace the present design rather than be explained away.

Acknowledgements

This working draft was developed as part of the open Experience Learning Layer project. Future versions will list contributors according to documented authorship and contribution criteria.

References

Alchourrón, Carlos E., Peter Gärdenfors, and David Makinson. 1985. 'On the Logic of Theory Change: Partial Meet Contraction and Revision Functions'. Journal of Symbolic Logic 50 (2): 510–30. https://doi.org/10.2307/2274239.

Bartlett, Frederic C. 1932. Remembering: A Study in Experimental and Social Psychology. Cambridge University Press.

Behrouz, Ali, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. 2025. 'Nested Learning: The Illusion of Deep Learning Architectures'. https://arxiv.org/abs/2512.24695.

Behrouz, Ali, Peilin Zhong, and Vahab Mirrokni. 2025. 'Titans: Learning to Memorize at Test Time'. https://arxiv.org/abs/2501.00663.

Chhikara, Prateek, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. 'Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory'. https://arxiv.org/abs/2504.19413.

Chen, Zhikai, Jialiang Gu, Junyu Yin, Xianxuan Long, Shenglai Zeng, Xiaoze Liu, Kai Guo, Keren Zhou, and Jiliang Tang. 2026. 'Exploring Cross-Scenario Generality of Agentic Memory Systems: Diagnostics and a Strong Baseline'. https://arxiv.org/abs/2606.04315.

Clifford, James, and Tomas Isakowitz. 1992. 'On the Semantics of Transaction Time and Valid Time in Bitemporal Databases'. NYU Working Paper IS-92-42. https://ssrn.com/abstract=1289024.

de Kleer, Johan. 1986. 'An Assumption-Based TMS'. Artificial Intelligence 28 (2): 127–62. https://doi.org/10.1016/0004-3702(86)90080-9.

Doyle, Jon. 1979. 'A Truth Maintenance System'. Artificial Intelligence 12 (3): 231–72. https://doi.org/10.1016/0004-3702(79)90008-0.

Fei, Tianxiang, Mingyang Song, Mao Zheng, and Xiang Yu. 2026. 'Memory Beyond Recall: A Dual-Process Cognitive Memory System for Self-Evolving LLM Agents'. https://arxiv.org/abs/2606.09483.

He, Zexue, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, et al. 2026. 'MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks'. https://arxiv.org/abs/2602.16313.

Hu, Yuanzhe, Yu Wang, and Julian McAuley. 2025. 'Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions'. https://arxiv.org/abs/2507.05257.

Hu, Qisheng, Quanyu Long, and Wenya Wang. 2026. 'When Continual Learning Moves to Memory: A Study of Experience Reuse in LLM Agents'. https://arxiv.org/abs/2604.27003.

Hu, Sen, Yuxiang Wei, Jiaxin Ran, Zhiyuan Yao, Xueran Han, Huacan Wang, Ronghao Chen, and Lei Zou. 2026. 'Does Memory Need Graphs? A Unified Framework and Empirical Analysis for Long-Term Dialog Memory'. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics, 26758–82. https://aclanthology.org/2026.acl-long.1232/.

Huo, Yupeng, Yaxi Lu, Zhong Zhang, Haotian Chen, and Yankai Lin. 2026. 'AtomMem: Learnable Dynamic Agentic Memory with Atomic Memory Operation'. https://arxiv.org/abs/2601.08323.

Jin, Yuxin, Siyuan Zhang, Hanchen Wang, Lu Qin, Ying Zhang, and Wenjie Zhang. 2026. 'EXG: Self-Evolving Agents with Experience Graphs'. https://arxiv.org/abs/2605.17721.

Jiang, Eric Hanchen, Zhi Zhang, Yuchen Wu, Levina Li, Dong Liu, Xiao Liang, Rui Sun, Yubei Li, Edward Sun, Haozheng Luo, Zhaolu Kang, Aylin Caliskan, Kai-Wei Chang, and Ying Nian Wu. 2026. 'Memory as a Controlled Process: Learned Adaptive Memory Management for LLM Agents'. https://arxiv.org/abs/2607.13591.

Kumaran, Dharshan, Demis Hassabis, and James L. McClelland. 2016. 'What Learning Systems Do Intelligent Agents Need?

Complementary Learning Systems Theory Updated'. Trends in Cognitive Sciences 20 (7): 512–34. https://doi.org/10.1016/j.tics.2016.05.004.

Letta. 2026. 'Letta: Platform for Building Stateful Agents'. Software repository. https://github.com/letta-ai/letta.

Latimer, Christopher, Nicolò Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. 2026. 'Hindsight: Structured Agent Memory that Retains, Recalls, and Reflects'. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, 275–85. https://doi.org/10.18653/v1/2026.acl-demo.27.

Lebo, Timothy, Satya Sahoo, and Deborah McGuinness, eds. 2013. 'PROV-O: The PROV Ontology'. W3C Recommendation. https://www.w3.org/TR/prov-o/.

Li, Yifei, Weidong Guo, Lingling Zhang, Rongman Xu, Muye Huang, Hui Liu, Lijiao Xu, Yu Xu, and Jun Liu. 2026. 'LoCoMo-Plus: Beyond-Factual Cognitive Memory Evaluation Framework for LLM Agents'. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics, 25085–100. https://doi.org/10.18653/v1/2026.acl-long.1150.

Lin, Huawei, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. 2026. 'MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation'. https://arxiv.org/abs/2605.27366.

Luo, Jinghao, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. 2026. 'From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms'. In Findings of the Association for Computational Linguistics: ACL 2026, 41622–52. https://doi.org/10.18653/v1/2026.findings-acl.2069.

Maharana, Adyasha, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. 'Evaluating Very Long-Term Conversational Memory of LLM Agents'. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 13851–70. https://doi.org/10.18653/v1/2024.acl-long.747.

McClelland, James L., Bruce L. McNaughton, and Randall C. O'Reilly. 1995. 'Why There Are Complementary Learning Systems in the Hippocampus and Neocortex: Insights from the Successes and Failures of Connectionist Models of Learning and Memory'.

Psychological Review 102 (3): 419–57. https://doi.org/10.1037/0033-295X.102.3.419.

Ornith Team. 2026. 'Ornith-1.0: Self-Scaffolding LLMs for Agentic Coding'. Technical release note, June 2026. https://ornith.ai/ornith_1_0.html.

Ouyang, Siru, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, et al. 2026. 'ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory'. In International Conference on Learning Representations. https://arxiv.org/abs/2509.25140.

Packer, Charles, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. 'MemGPT: Towards LLMs as Operating Systems'. https://arxiv.org/abs/2310.08560.

Park, Joon Sung, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. 'Generative Agents: Interactive Simulacra of Human Behavior'. In Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, 1–22. https://doi.org/10.1145/3586183.3606763.

Paul, Swarna Kamal, Shubhendu Sharma, and Nitin Sareen. 2026. 'GAAMA: Graph Augmented Associative Memory for Agents'. https://arxiv.org/abs/2603.27910.

Rasmussen, Preston, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. 'Zep: A Temporal Knowledge Graph Architecture for Agent Memory'. https://arxiv.org/abs/2501.13956.

Ren, Tao, Weiyao Luo, Hui Yang, Rongzhi Zhu, Xiang Huang, Yuchuan Wu, Bingxue Chou, et al. 2026. 'Scaling Self-Evolving Agents via Parametric Memory'. https://arxiv.org/abs/2606.04536.

Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. 'Reflexion: Language Agents with Verbal Reinforcement Learning'. In Advances in Neural Information Processing Systems. Vol. 36. https://arxiv.org/abs/2303.11366.

Shen, Yiting, Kun Li, Wei Zhou, and Songlin Hu. 2026. 'Mem2ActBench: A Benchmark for Evaluating Long-Term Memory Utilization in Task-Oriented Autonomous Agents'. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics, 8173–90. https://doi.org/10.18653/v1/2026.acl-long.370.

Shukla, Navnit, Kamal Pandey, and Omsankar Tiwari. 2026. 'TurboVec: A Case Study in Cost-Efficient Private Retrieval for Enterprise RAG via Codebook-Oblivious Quantization'. https://arxiv.org/abs/2607.16973.

Su, Miao, Yucan Guo, Zhongni Hou, Long Bai, Zixuan Li, Yufei Zhang, Guojun Yin, et al. 2026. 'Beyond Dialogue Time: Temporal Semantic Memory for Personalized LLM Agents'. https://arxiv.org/abs/2601.07468.

Sumers, Theodore R., Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2024. 'Cognitive Architectures for Language Agents'.

Transactions on Machine Learning Research. https://arxiv.org/abs/2309.02427.

Tan, Haoran, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. 'MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-Based Agents'. In Findings of the Association for Computational Linguistics: ACL 2025. https://arxiv.org/abs/2506.21605.

Tan, Zhen, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long T. Le, Yiwen Song, et al. 2025. 'In Prospect and Retrospect: Reflective Memory Management for Long-Term Personalized Dialogue Agents'. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, 8416–39. https://doi.org/10.18653/v1/2025.acl-long.413.

TencentDB Agent Memory Team. 2026. 'TencentDB Agent Memory'. Software repository, accessed 9 August 2026. https://github.com/TencentCloud/TencentDB-Agent-Memory.

Tulving, Endel. 1972. 'Episodic and Semantic Memory'. In Organization of Memory, edited by Endel Tulving and Wayne Donaldson, 381–403. Academic Press.

Wang, Guanzhi, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023.

'Voyager: An Open-Ended Embodied Agent with Large Language Models'. In Transactions on Machine Learning Research. https://arxiv.org/abs/2305.16291.

Wang, Zora Zhiruo, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. 'Agent Workflow Memory'. In Proceedings of the 42nd International Conference on Machine Learning, 267:63897–911. Proceedings of Machine Learning Research.

Wu, Di, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. 'LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory'. In International Conference on Learning Representations. https://arxiv.org/abs/2410.10813.

Xu, Wujiang, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. 'A-MEM: Agentic Memory for LLM Agents'.

In Advances in Neural Information Processing Systems. https://arxiv.org/abs/2502.12110.

Yang, Ke, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. 2026.

'PlugMem: A Task-Agnostic Plugin Memory Module for LLM Agents'. https://arxiv.org/abs/2603.03296.

Yu, Yi, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026. 'Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents'. https://arxiv.org/abs/2601.01885.

Zhang, Jiawen, Kejia Chen, Jiachen Ma, Yangfan Hu, Lipeng He, Yechao Zhang, Jian Liu, Xiaohu Yang, Tianwei Zhang, and Ruoxi Jia. 2026. 'Beyond Similarity: Trustworthy Memory Search for Personal AI Agents'. https://arxiv.org/abs/2606.06054.

Zhang, Haozhen, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. 'MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents'. https://arxiv.org/abs/2602.02474.

Zhao, Andrew, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. 'ExpeL: LLM Agents Are Experiential Learners'. In Proceedings of the AAAI Conference on Artificial Intelligence, 38:19632–42. 17. https://doi.org/10.1609/aaai.v38i17.29936.

Zhong, Wanjun, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. 'MemoryBank: Enhancing Large Language Models with Long-Term Memory'. In Proceedings of the AAAI Conference on Artificial Intelligence, 38:19724–31. 17. https://doi.org/10.1609/aaai.v38i17.29946.

Zhou, Huichi, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, et al. 2026. 'Memento-Skills: Let Agents Design Agents'. https://arxiv.org/abs/2603.18743.

Zep. 2026. 'Graphiti: Build Real-Time Knowledge Graphs for AI Agents'. Software repository. https://github.com/getzep/graphiti.